

Saga Pattern

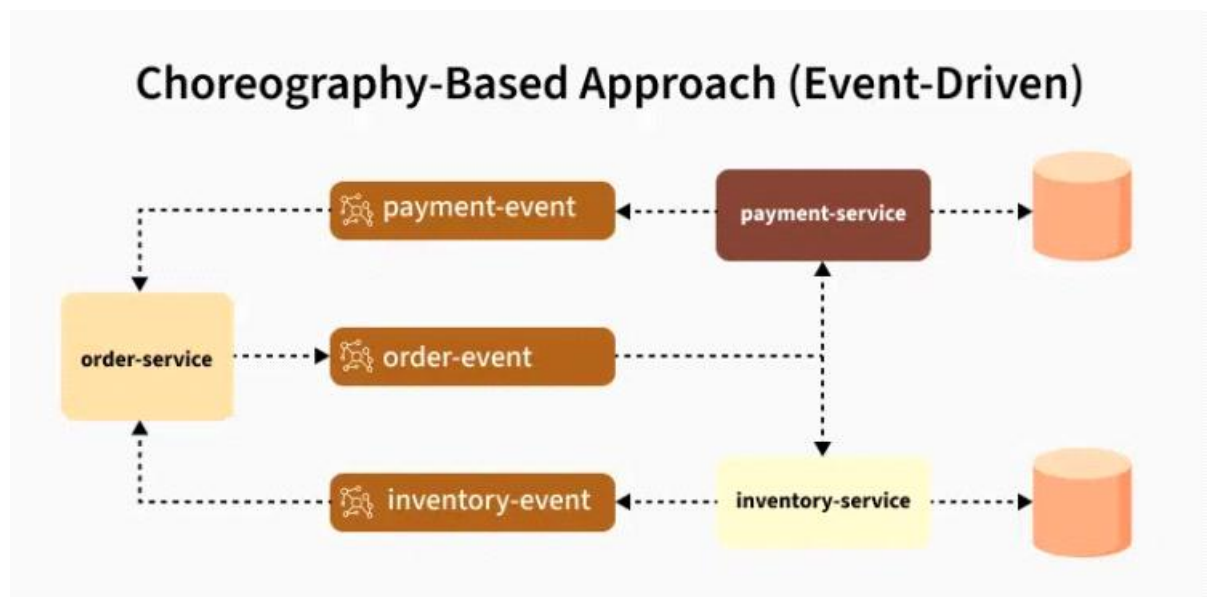
22 January 2026 13:42

SAGA

The SAGA Design Pattern is a pattern used to **manage long-running and distributed transactions**, particularly in microservices architecture. Unlike traditional monolithic transactions, which require a single, centralized transaction management system, the SAGA pattern breaks down a complex transaction into a series of smaller, isolated operations, each handled by a different service. **SAGA** addresses the limitations of **2PC** (2 phase commit) by breaking a transaction into smaller, independent steps, each with its own compensating action if something goes wrong. Here's how it solves key issues of 2PC:

Approaches to Implement SAGA Design Pattern

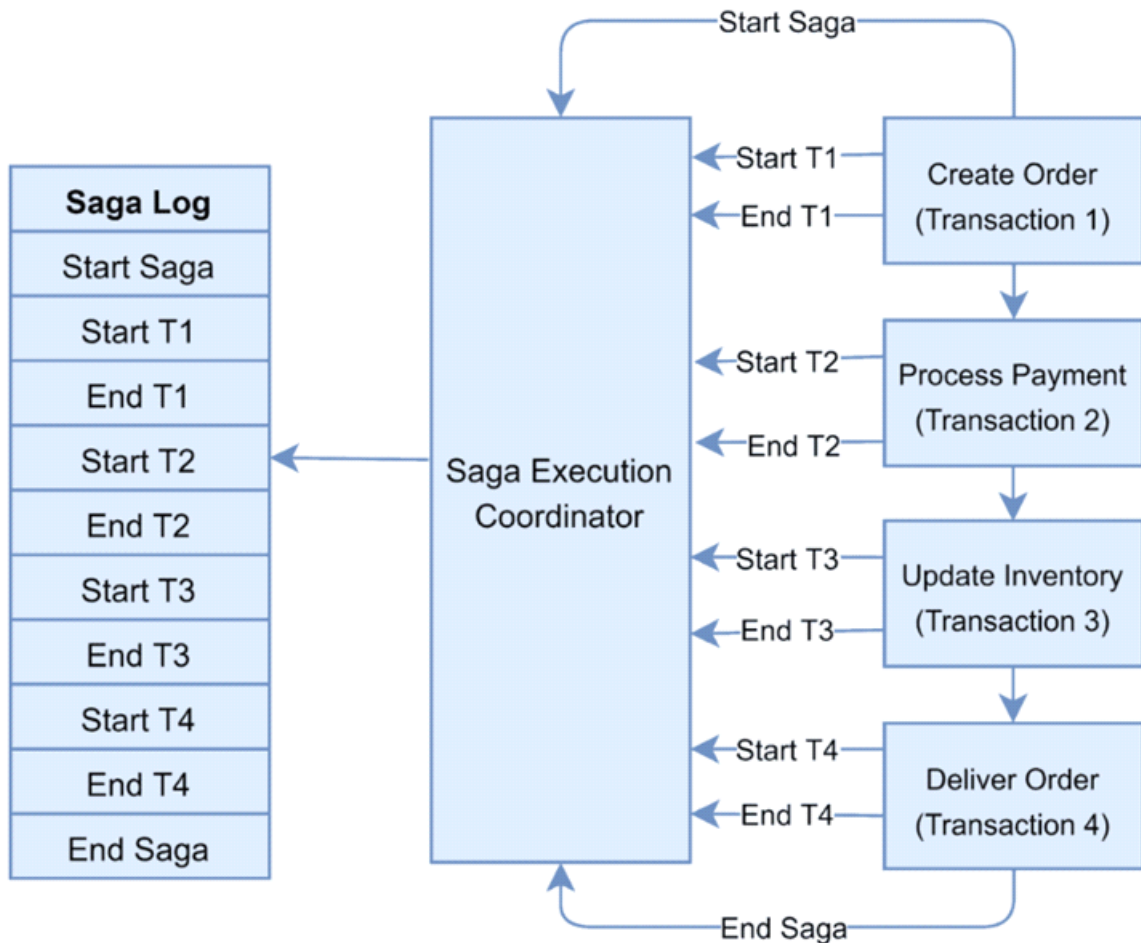
- Choreography-Based Approach (Event-Driven)



- Orchestration-Based Approach (Centralized)

For any failure, the SEC component inspects the Saga log to identify the

impacted components and the sequence in which the compensating transactions should run. It can then identify the transactions successfully rolled back, which ones are pending, and can take appropriate actions:



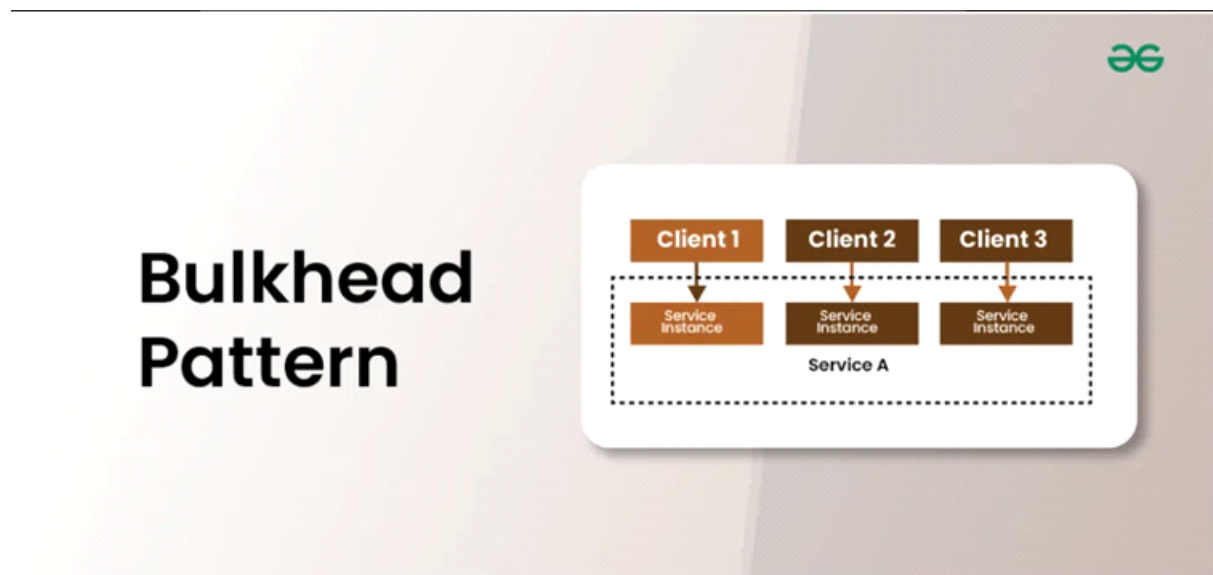
Reference Links:

- <https://www.geeksforgeeks.org/system-design/saga-design-pattern/>
- <https://www.baeldung.com/cs/saga-pattern-microservices>
- <https://microservices.io/patterns/data/saga.html>

Bulkhead Pattern

The Bulkhead Pattern is a design principle used in software architecture to improve system [resilience](#) by isolating components or resources within a system. It is named after the watertight compartments ("bulkheads") on ships, which prevent flooding in one area from affecting the entire vessel.

- In software, the Bulkhead Pattern involves partitioning components or resources into separate "bulkheads" to limit the impact of failures or overloads in one area on the rest of the system.
- This isolation helps prevent cascading failures and ensures that a failure in one part of the system does not bring down the entire system.
- Common implementations of the Bulkhead Pattern include using separate thread pools, processes, or containers to isolate and manage resources for different components or services within a system.



Example of Bulkhead Implementation

An example of Bulkhead Pattern implementation can be seen in a web application where different types of tasks are processed by separate thread pools to ensure fault isolation and prevent resource contention.

Types of Bulkheads in Software Systems

In software systems, bulkheads are implemented to isolate components, resources, or processes from one another to enhance resilience and fault tolerance. Different types of bulkheads can be employed based on the specific requirements and characteristics of the system. Here are some common types of bulkheads in software systems:

1. Thread Pool Bulkhead

- In multithreaded applications, a thread pool bulkhead involves allocating separate thread pools for different types of tasks or operations. For example, user-facing requests may be processed by one thread pool, while background processing tasks are handled by another.
- This isolation prevents resource contention and ensures that failures or performance issues in one thread pool do not affect the operation of others.

2. Service Bulkhead

- In distributed systems, a service bulkhead involves isolating services or microservices from one another to prevent cascading failures.
- Each service operates independently and has its own resources and dependencies. This isolation helps contain faults within individual services and prevents failures from propagating across the entire system.

3. Database Bulkhead

- In database-intensive applications, a database bulkhead involves partitioning databases or database connections to isolate different types of data or workload. For example, read-heavy and write-heavy operations may be directed to separate database instances or partitions.
- This isolation prevents performance bottlenecks and ensures that failures or slowdowns in one database do not impact other database operations.

4. Network Bulkhead

- In networked applications, a network bulkhead involves segregating network traffic or communication channels to isolate different types of data or services.
- For example, high-priority traffic may be routed through dedicated network paths, while low-priority traffic is routed through separate paths. This isolation helps prioritize critical traffic and prevent congestion or failures from affecting other network activities.

5. Process Bulkhead

- In process-based architectures, a process bulkhead involves running separate processes or containers to isolate different components or services.
- Each process operates within its own runtime environment and has its own resources and dependencies. This isolation helps contain faults within individual processes and prevents failures from affecting other parts of the system.

6. Resource Bulkhead

- In resource-intensive applications, a resource bulkhead involves

partitioning resources such as CPU, memory, or storage to prevent overutilization and ensure fair resource allocation.

- For example, CPU cores may be assigned to specific tasks or services to prevent one task from monopolizing resources and starving others

Challenges of Bulkhead Implementation

Implementing the Bulkhead Pattern in software systems can pose several challenges, which require careful consideration and mitigation strategies. Here are some common challenges associated with bulkhead implementation:

- **Complexity:**

- Designing and implementing bulkheads can introduce complexity to the system architecture, especially in distributed or microservices-based systems.
- Managing interactions, dependencies, and communication between bulkheads requires careful coordination and may increase development and maintenance overhead.

- **Resource Management:**

- Allocating and managing resources (such as threads, memory, CPU, database connections) for each bulkhead can be challenging, particularly in dynamic or heterogeneous environments.
- Balancing resource usage, preventing resource contention, and optimizing resource allocation across bulkheads requires sophisticated monitoring and management mechanisms.

- **Overhead and Latency:**

- Introducing bulkheads may incur additional overhead and latency due to the overhead of isolation mechanisms, communication between bulkheads, and coordination of resources.
- Minimizing overhead while maintaining effective isolation and fault tolerance is a delicate balancing act that requires careful optimization and tuning.

- **Synchronization and Consistency:**

- Ensuring synchronization and consistency between bulkheads, especially in distributed systems, can be challenging.
- Coordinating concurrent access to shared resources, maintaining data consistency across bulkheads, and handling distributed transactions require robust synchronization mechanisms and may introduce performance bottlenecks.

- **Scalability and Elasticity:**

- Scaling bulkheads dynamically to accommodate changing workload demands or resource requirements can be complex, particularly in

environments with heterogeneous or distributed infrastructure.

- Implementing scalable and elastic architectures that can automatically adjust resource allocation and rebalance workloads across bulkheads is a significant challenge.

Circuit Breaker Pattern

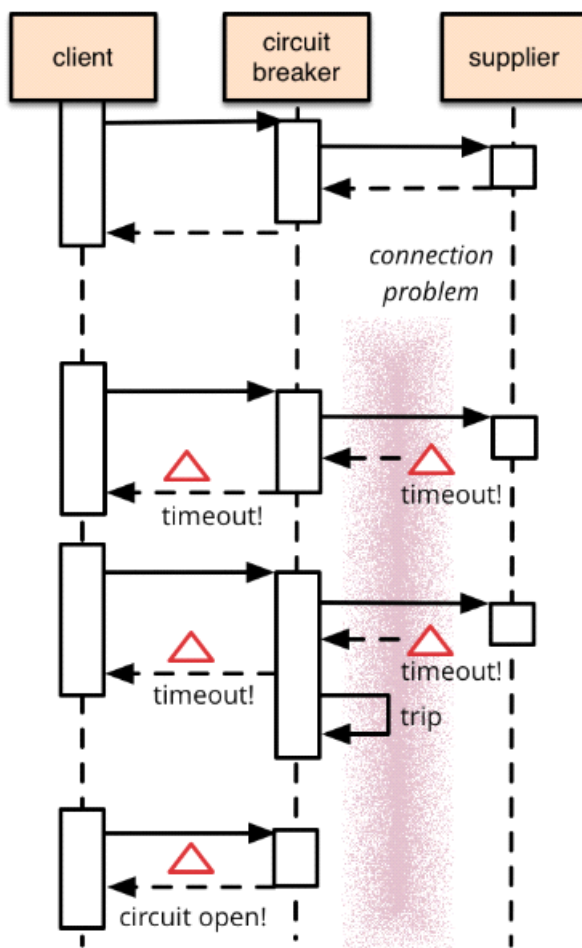
22 January 2026 13:42

Circuit Breaker

The [Circuit Breaker pattern](#) helps us in preventing a cascade of failures when a remote service is down.

After a number of failed attempts, we can consider that the service is unavailable/overloaded and eagerly reject all subsequent requests to it. In this way, we can save system resources for calls which are likely to fail.

The basic idea behind the circuit breaker is very simple. You wrap a protected function call in a circuit breaker object, which monitors for failures. Once the failures reach a certain threshold, the circuit breaker trips, and all further calls to the circuit breaker return with an error, without the protected call being made at all. Usually you'll also want some kind of monitor alert if the circuit breaker trips.



A *CircuitBreaker* can be in one of the three states:

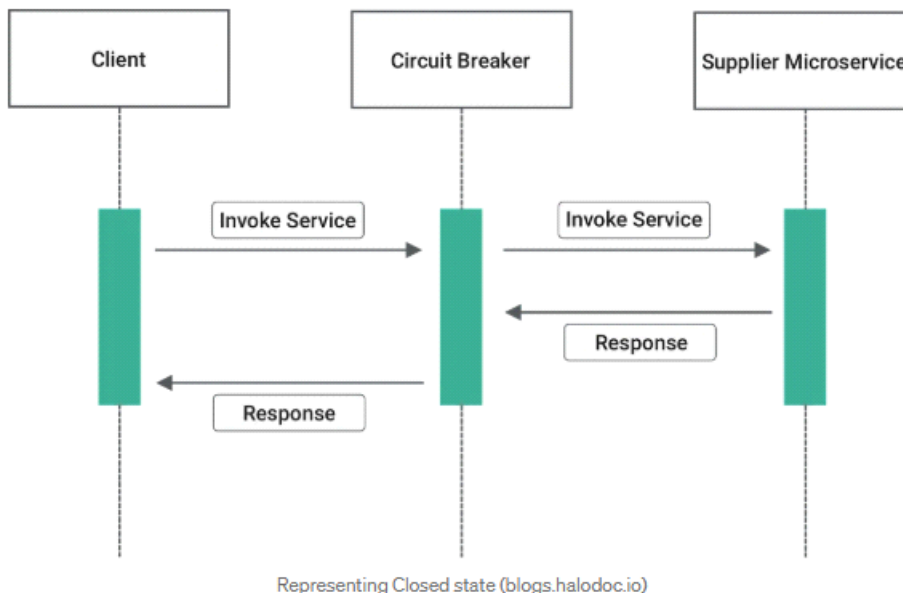
- *CLOSED* – everything is fine, no short-circuiting involved
- *OPEN* – remote server is down, all requests to it are short-circuited
- *HALF_OPEN* – a configured amount of time since entering *OPEN* state has elapsed and *CircuitBreaker* allows requests to check if the remote service is back online

We can configure the following settings:

- the failure rate threshold above which the *CircuitBreaker* opens and starts short-circuiting calls
- the wait duration which defines how long the *CircuitBreaker* should stay open before it switches to half open
- the size of the ring buffer when the *CircuitBreaker* is half open or closed
- a custom *CircuitBreakerEventListener* which handles *CircuitBreaker* events
- a custom *Predicate* which evaluates if an exception should count as a failure and thus increase the failure rate

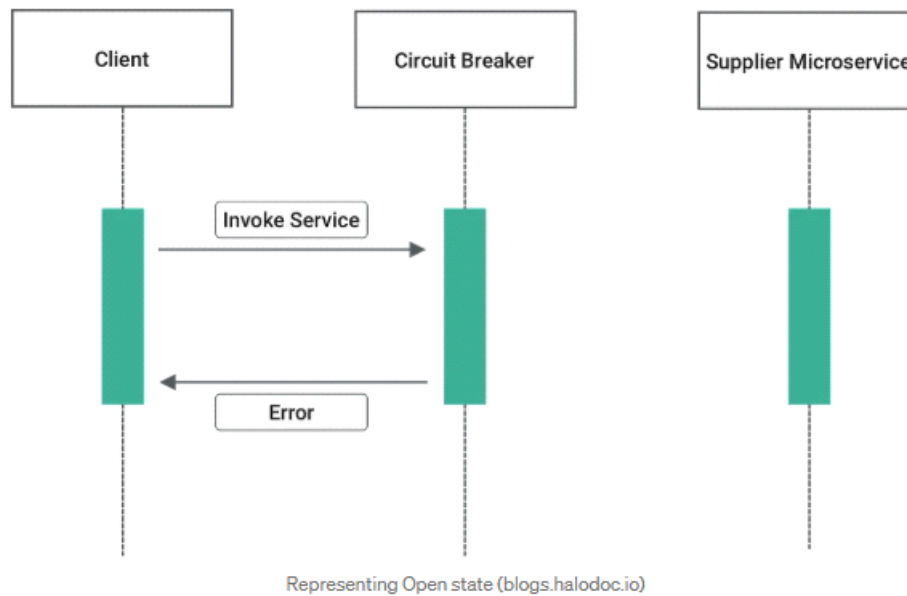
Closed state

In this state, the Circuit Breaker routes the requests to the Microservice and counts the number of failures in each period of time. That means it work without any failures. But if the number of failures in a certain period of time exceeds a threshold, the circuit will trip and will move to an “Open” state.



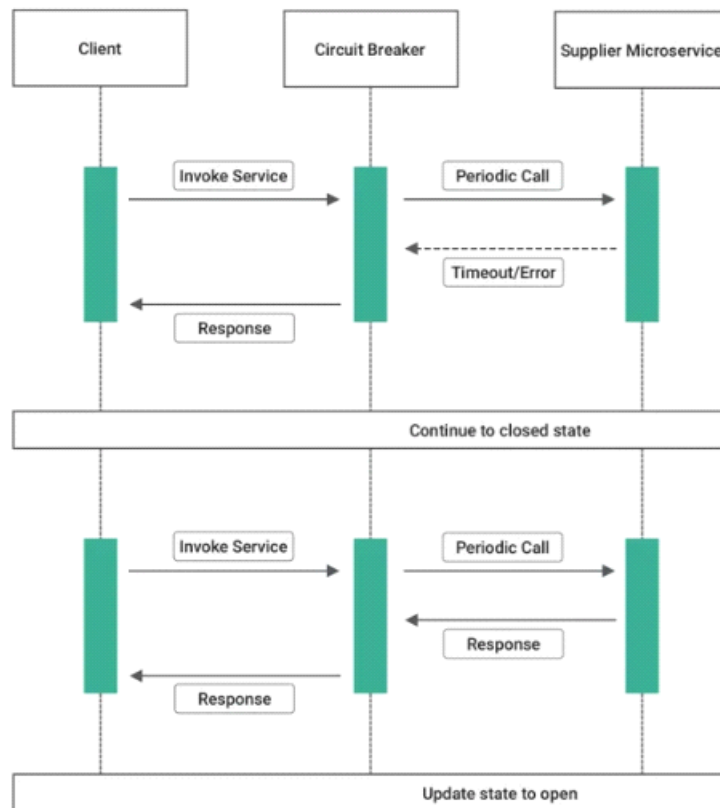
Open state

When Circuit breaker moves to the “Open” state, requests from the Microservices will fail immediately, and an exception will be returned. However, after a timeout, the Circuit Breaker will go to the “Half-Open” state.



Half-Open state

In this state, the Circuit Breaker allows only a limited number of requests from the Microservice, to pass through and invoke the operation. If these requests are successful, the Circuit Breaker will go back to the “Closed” state. However, if any request fails again, it goes back to the “Open” state.



Representing Half-open state (blogs.halodoc.io)



Resilience4j vs Hystrix — Key Differences



1. Project Status

Hystrix (Netflix)

- **Status:** Deprecated / Maintenance mode since 2018
- No new features.
- Spring Cloud removed Hystrix in newer versions.

Resilience4j

- **Fully active**, modern, and maintained.
- Officially recommended by **Spring Cloud** for circuit breakers.

✓ 2. Architecture & Design Philosophy

Resilience4j

- Built **specifically for Java 8+** functional programming style.
- Lightweight **modular** architecture:
 - Circuit Breaker
 - Retry
 - Rate Limiter
 - Bulkhead
 - TimeLimiter
 - Cache
 - Fallbacks

Hystrix

- Heavy, monolithic library.
- Implements only:
 - Circuit Breaker
 - Bulkhead (via thread pools)
 - Fallback

The Spring Cloud Netflix project is really just an annotation-based wrapper library around Hystrix. Therefore, these two libraries are tightly-coupled. This means we can't switch to another circuit breaker implementation without changing the application.

The Spring Cloud Circuit Breaker project solves this. **It provides an abstraction layer across different circuit breaker implementations.** It's a pluggable architecture. So, we can code against the provided abstraction/interface and switch to another implementation based on our needs.

CQRS

☑ What is CQRS?

CQRS (Command Query Responsibility Segregation) is a design pattern that *separates read operations and write operations into different models*, instead of using a single model or service for both.

- **Commands** → modify data
(create, update, delete)
- **Queries** → read data
(get flight status, list pilots, show dashboard)

The key idea:

The model used to update data should not be the same as the model used to read data.

☑ Why CQRS?

1. **Read and write workloads scale independently**
(In your project, reads are *huge* due to dashboards, monitoring, tracking)
2. **Different models per use case**
Commands → validated, transactional models
Queries → denormalized, fast-read models (Redis, Elastic)
3. **Real-time updates become easier**
Because state changes publish events → WebSockets, Kafka, Angular UI
4. **Perfect for microservices + event-driven architecture**

💡 Benefits of CQRS

1. Independent scaling

- Write side handles lower volume but heavy logic
- Read side handles high volume and fast queries
→ Each can be scaled separately.

2. Different data models

- Write side uses normalized relational schemas
- Read side can be denormalized, cached, or optimized (Redis, Elastic)

3. Better performance

Reads become extremely fast because the data is shaped for the UI.

4. Event-driven integration

CQRS naturally works with **events**, which allows:

- real-time updates
- audit trails
- microservice integration

⚙️ How CQRS Works (Simple Theory)

Write Process — Commands

1. A command comes in (e.g., *AssignCrew*, *DelayFlight*)
2. The command side:
 - validates business rules
 - updates the write database
 - generates an event (“CREW_ASSIGNED”, “FLIGHT_DELAYED”)

Read Process — Queries

1. Query side gets the event
2. Updates a separate **read model**, often:
 - Redis
 - Elasticsearch
 - MongoDB
3. The UI reads from this optimized model

🎯 When to Use CQRS

Use CQRS when:

- read and write workloads differ drastically
- real-time dashboards or monitoring is required
- business logic is complex
- system uses microservices and event-driven architecture

Do **NOT** use CQRS when the system is small and simple — too much overhead.

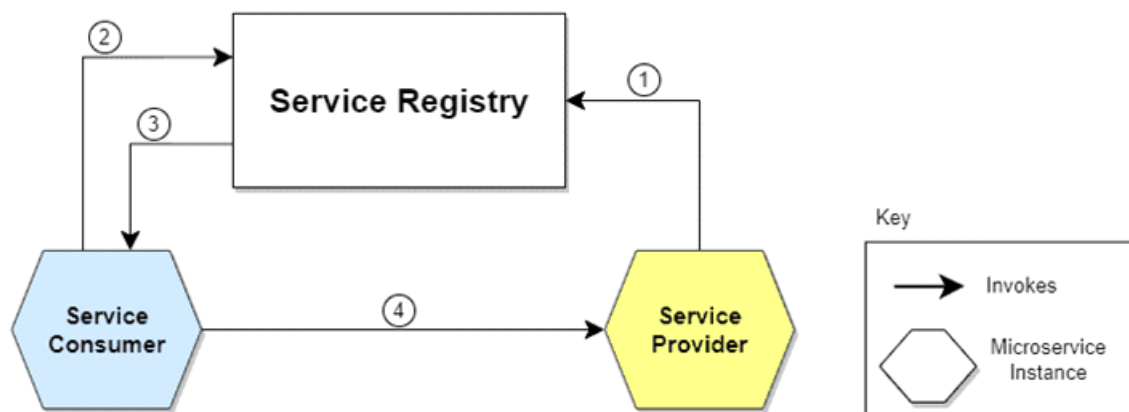
SERVICE DISCOVERY

A [microservices-based application](#) typically runs in virtualized or containerized environments. The number of instances of a service and its locations changes dynamically. We need to know where these instances are and their names to allow requests to arrive at the target microservice. This is where tactics such as Service Discovery come into play.

The Service Discovery mechanism helps us know where each instance is located. In this way, a Service Discovery component acts as a registry in which the addresses of all instances are tracked. The instances have dynamically assigned network paths. Consequently, if a client wants to make a request to a service, it must use a Service Discovery mechanism.

A microservice needs to know the location (IP address and port) of every service it communicates with. If we don't employ a Service Discovery mechanism, service locations become coupled, leading to a system that's difficult to maintain

Service Discovery handles things in two parts. First, it provides a mechanism for an instance to register and say, “I’m here!” Second, it provides a way to find the service once it has registered.

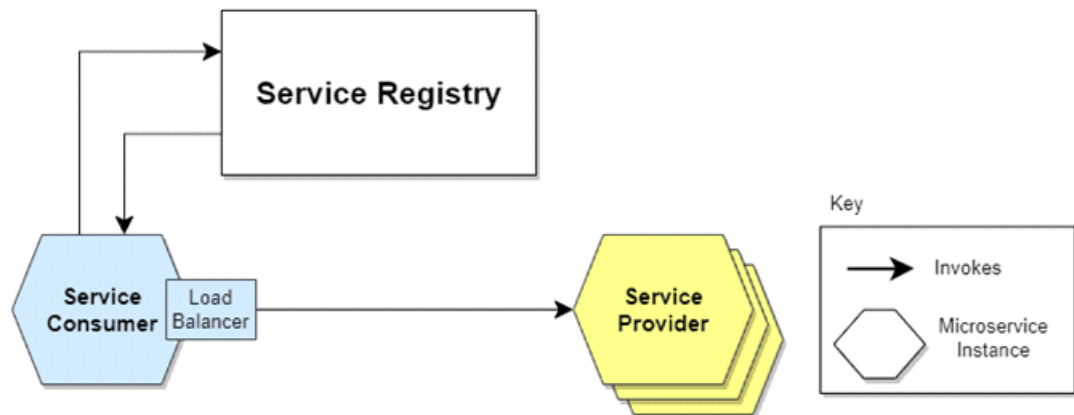


Service Discovery Implementations

1) Client-Side Service Discovery

When using Client-Side Discovery, **the Service Consumer is responsible for determining the network locations of available service instances and load balancing requests between them.** The client queries the Service Register. Then the client uses a load-balancing algorithm to choose one of the available service instances and performs a request.

The following diagram shows the pattern just described:

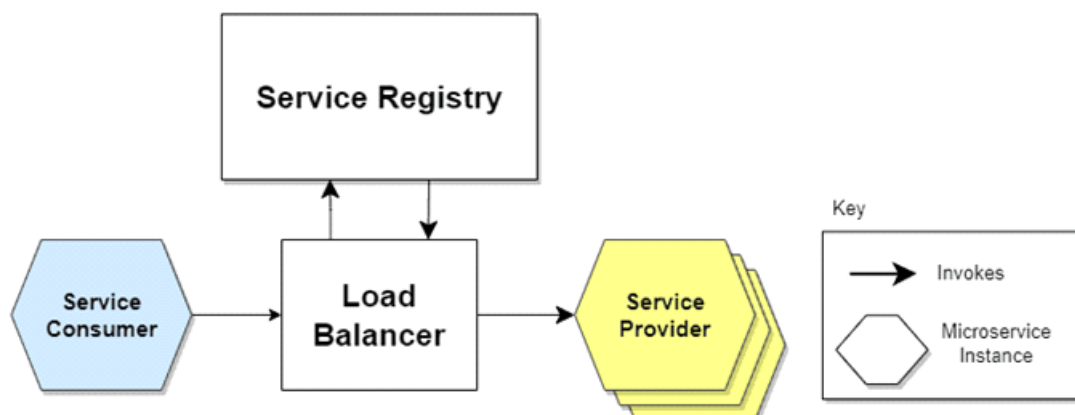


Giving responsibility for client-side load balancing is both a burden and an advantage. **It's an advantage because it saves an extra hop that we would've had with a dedicated load balancer. It's a disadvantage because the Service Consumer must implement the load balancing logic.**

2) Server-Side Service Discovery

The alternate approach to Service Discovery is the Server-Side Discovery model, which uses an intermediary that acts as a [Load Balancer](#). The client makes a request to a service via a load balancer that acts as an orchestrator. The load balancer queries the Service Registry and routes each request to an available service instance.

The following diagram shows how communication takes place:



In this approach, a dedicated actor, the Load Balancer, does the job of load balancing. This is the main advantage of this approach. Indeed, **creating this**

level of abstraction makes the Service Consumer lighter, as it doesn't have to deal with the lookup procedure.

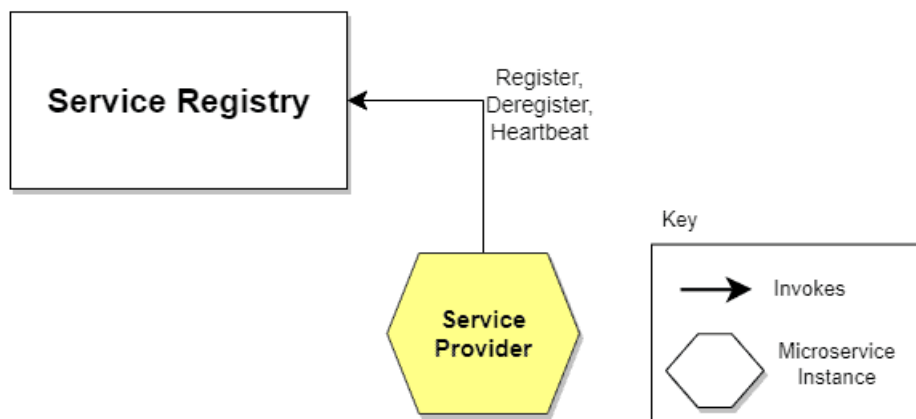
What Is Service Registry?

The Service Register is a crucial part of service identification. **It's a database containing the network locations of service instances.** A Service Registry must be highly available and up-to-date. Clients can cache the network paths obtained from the Service Registry; however, this information eventually becomes obsolete, and clients won't reach the service instances. Consequently, a Service Registry consists of a cluster of servers that use a replication protocol to maintain consistency.

Service Registration Options

a. Self-Registration

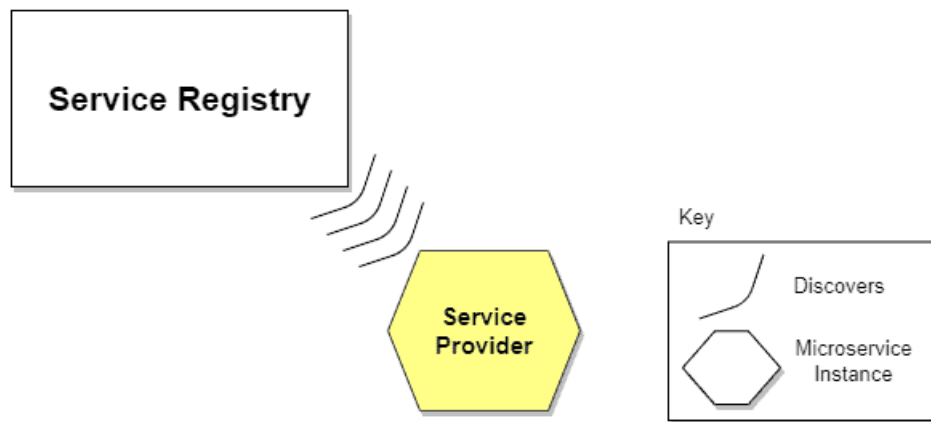
When using the self-registration model, a service instance is responsible for registering and de-registering itself in the Service Registry. In addition, if necessary, a service instance sends [heartbeat](#) requests to keep its registration alive. The following diagram shows the structure of this pattern:



The self-registration model has several pros and cons. One advantage is that it's relatively simple and doesn't require other system components as intermediaries. However, a significant disadvantage is that it couples service instances to the Service Registry, which means we must implement the registration code in each language and framework used.

b. Third-party Registration

When using the third-party registration model, the service instances aren't responsible for registration in the Service Registry. Instead, another system component known as the Service Register is responsible for registration. **The Service Register keeps track of changes to running instances by polling the deployment environment or subscribing to events.** When it detects a newly available service instance, it records it in its database. The Service Registry also de-registers terminated service instances. The following diagram illustrates this:



One of the main advantages is that services are decoupled from the Service Registry. There's no need to implement service registration logic for each programming language and framework. Instead, the registration of service instances is managed centrally within a dedicated service.

One disadvantage of this model is that, unless it's embedded in the deployment environment, it's yet another highly available system component that needs to be set up and managed.



What Does Spring Cloud Use by Default?

Before Kubernetes era:



Eureka was the default and most widely used.

In modern cloud deployments:



Kubernetes DNS + Spring Cloud Kubernetes is preferred.

Comparison Table

Discovery Mechanism	Type	Strong Point	Typical Use Case
Eureka	Client-side registry	Spring-first integration	Java microservices
Consul	Distributed registry	Multi-platform, K/V, health checks	Hybrid cloud environments
Zookeeper	CP registry	Strong consistency	Enterprise coordination services
Kubernetes DNS/API	Platform-native	No extra registry needed	Cloud-native deployments
AWS Cloud Map	Cloud-native	AWS-integrated discovery	EKS/ECS microservices

RETRY

What is the Retry Pattern?

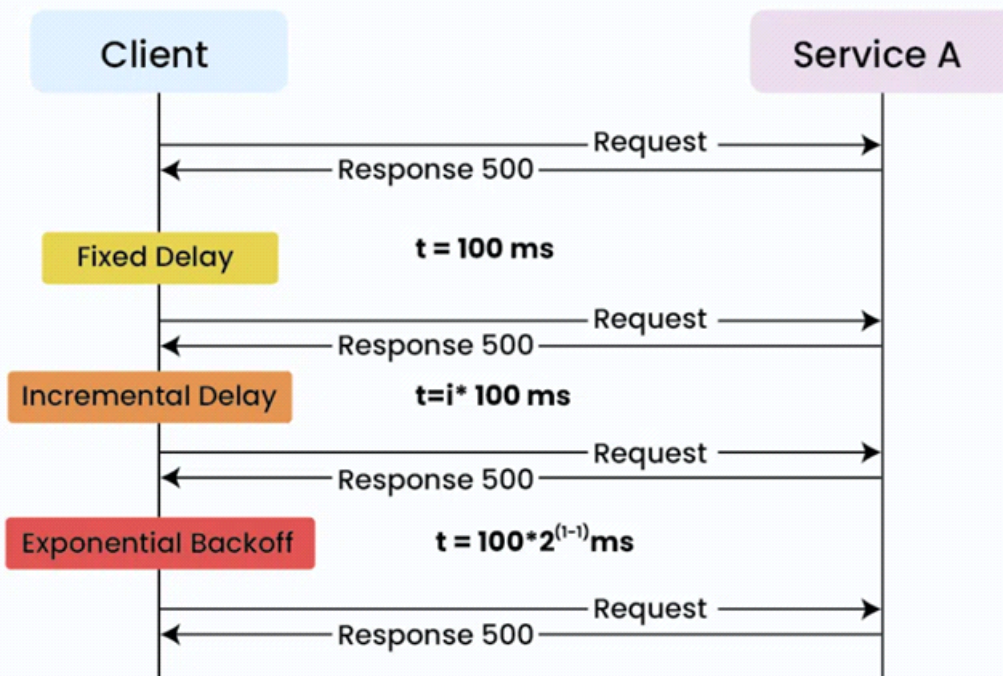
The Retry Pattern in microservices system design is a strategy used to handle temporary failures that occur during service communication. In a distributed system where microservices interact with each other, transient issues such as network interruptions or brief service outages can lead to failed requests. To address these transient failures, the Retry Pattern involves automatically retrying a failed request a predetermined number of times before considering it a permanent failure.

- This approach helps ensure that brief disruptions do not result in service failures or degraded performance.
- The pattern typically includes configurable parameters, such as the number of retries and the delay between attempts, often incorporating exponential backoff and jitter to prevent overwhelming the system with retries.
- By implementing the Retry Pattern, microservices can maintain higher reliability and resilience, improving the overall robustness of the system.

How Retry Pattern Works?

The Retry Pattern in microservices works by automatically handling temporary failures in service communication through a series of retries. Here's a breakdown of how it functions:

How Retry Pattern Works?



1. **Initial Request:** When a microservice makes a request to another service, it may encounter transient errors due to issues like network interruptions or momentary service unavailability.
2. **Retry Logic:** If the initial request fails, the Retry Pattern triggers a retry mechanism. This involves automatically resending the request a specified number of times. The retry attempts are made according to a defined strategy, which may include constant, incremental, or exponential backoff periods between retries.
3. **Backoff Strategy:** To prevent overwhelming the system or causing further issues, retries are spaced out using a backoff strategy. Constant backoff uses a fixed delay between retries, while exponential backoff increases the delay progressively with each retry. Jitter (randomized delay) can also be added to spread out retry attempts and reduce the risk of multiple services retrying simultaneously.
4. **Retry Limits:** There are usually limits set on the number of retry attempts to avoid endless retry loops. Once the maximum number of retries is reached, the system may log an error, trigger an alert, or take alternative actions, such as notifying a user or falling back to a different service.
5. **Error Handling:** If the retries still fail, the system may handle the error according to predefined rules, such as retrying at a later time, using cached data, or gracefully degrading service functionality.

Real-World Examples

The Retry Pattern is widely used in real-world microservices architectures to handle transient failures and ensure reliability. Here are some practical examples:

- **Amazon's AWS Services:**

- AWS services, such as DynamoDB and S3, often encounter transient network issues or throttling.
- AWS SDKs include built-in retry mechanisms with configurable backoff strategies to handle such issues seamlessly.
- Ensures higher reliability and robustness for applications interacting with AWS services, minimizing the impact of temporary disruptions.
- **Netflix:**
 - Netflix uses the Retry Pattern in its microservices architecture to handle transient errors in its distributed systems.
 - Their resilience library, Hystrix, provides support for retries and circuit breaking, helping to manage service failures and maintain high availability.
 - Improves the reliability of their streaming service and enhances user experience by automatically handling temporary service disruptions.
- **Stripe:**
 - Stripe's payment processing system employs the Retry Pattern to handle transient errors during payment transactions.
 - For instance, network timeouts or temporary service unavailability are managed with retry logic to ensure successful transaction processing.
 - Ensures that payment transactions are completed reliably, even in the face of temporary network or service issues.

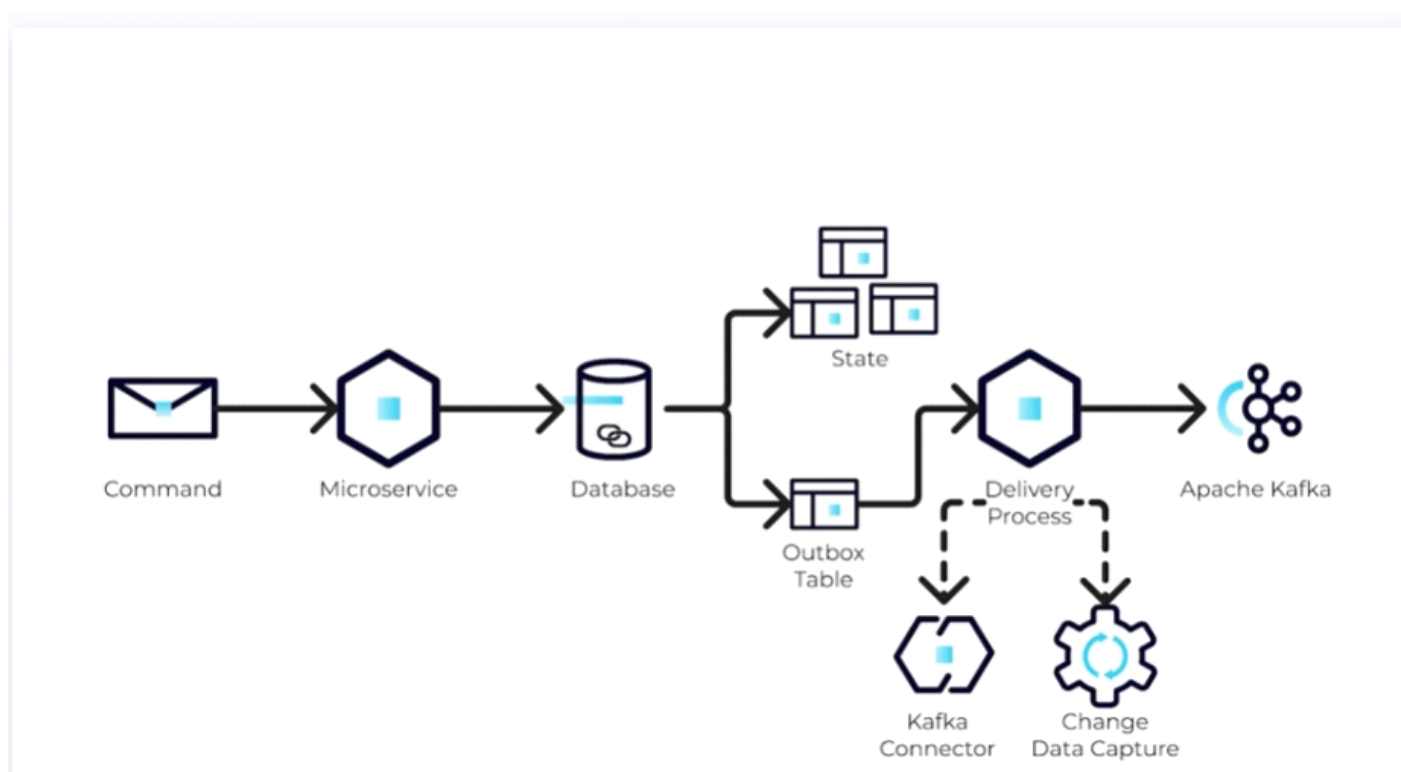
Outbox Pattern

22 January 2026 13:45

[What is the Transactional Outbox Pattern? | Designing Event-Driven Microservices](#)



The Outbox Pattern ensures reliable event publishing by storing events in the same database transaction as business data and publishing them asynchronously.



What problem does it solve?

The **Outbox Pattern** solves the **dual-write problem** in microservices.

How do you **update the database and publish an event/message** *reliably* without losing data or sending duplicates?

The core problem (dual write issue)

Without Outbox:

- Update database

- Publish event to Kafka

If step 2 fails after step 1:

- DB is updated
- Event is **never published**
- System becomes **inconsistent**

What is the Outbox Pattern?

The Outbox Pattern ensures **database changes and events are written atomically** in a single transaction.

Instead of publishing directly to Kafka:

- Write the event to an **outbox table**
- Publish later in a **reliable way**

How it works (step-by-step)

1 Business transaction

- Service updates its business table
- Service inserts an event into the **outbox table**
- Both happen in **one DB transaction**

☑ Either both succeed or both fail

2 Outbox processor

A separate process:

- Polls the outbox table
- Publishes events to Kafka (or other broker)
- Marks events as processed

```
Service
|
|-- DB Transaction
|   |-- Update Orders table
|   |-- Insert event into Outbox table
|
Outbox Processor
|
|-- Read Outbox
|-- Publish to Kafka
|-- Mark as sent
```

Real-world example

Example: Order Service

When an order is placed:

- Order is saved in DB
- OrderCreated event is written to outbox

Later:

- Outbox processor publishes OrderCreated to Kafka

- Inventory and Payment services consume it

Why not publish directly to Kafka?

Because:

- Kafka is external
- DB and Kafka do not share transactions
- Network failures can cause data loss

Outbox makes this **reliable and consistent**.

Delivery guarantees

Outbox pattern typically provides:

- **At-least-once delivery**

So:

- Consumers must be **idempotent**
- Duplicate events are possible

How events are published (two approaches)

1 Polling-based Outbox

- Background job polls DB
- Simple and common
- Slight delay

2 CDC-based Outbox (advanced)

- Tools like Debezium
- Reads DB transaction log
- Near real-time, no polling

Advantages

- ☒ Solves dual-write problem
- ☒ Strong consistency
- ☒ Works with any message broker
- ☒ Simple to reason about

Trade-offs

- ☒ Extra table & storage
- ☒ Slight latency
- ☒ More operational complexity
- ☒ Requires idempotent consumers

When should you use Outbox?

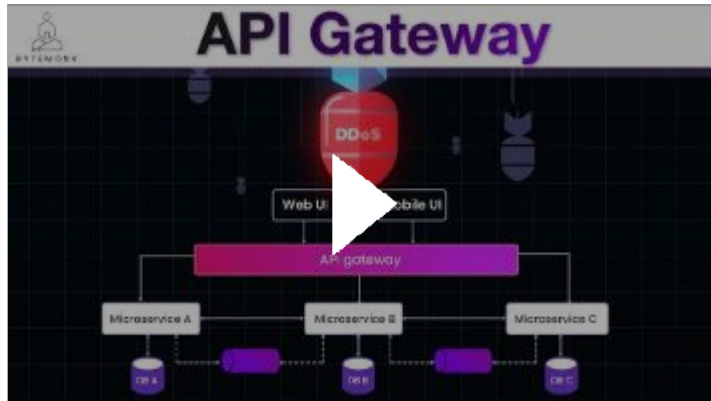
Use it when:

- You publish events after DB updates
- Data consistency is critical
- You use Kafka / RabbitMQ
- You want reliable event delivery

API Gateway Pattern

22 January 2026 13:42

[Designing with API Gateway: Microservices Unleashed](#)



What is a API Gateway?

An API Gateway provides a single entry point to microservices, handling routing, security, and cross-cutting concerns.

What problem does it solve?

The **API Gateway Pattern** solves the problem of **clients directly talking to many microservices**. It provides a **single entry point** for all client requests.

What is an API Gateway?

An **API Gateway** is a server that sits between clients and microservices, handling **request routing, security, and cross-cutting concerns**. Clients talk to the gateway — **not** individual services.

Why do we need it?

Without an API Gateway:

- Clients call many services
- Client logic becomes complex
- Security is duplicated everywhere

With an API Gateway:

- ☒ Centralized control
- ☒ Simplified clients
- ☒ Consistent behavior

How it works

- 1 Client sends request to API Gateway
- 2 Gateway authenticates & validates
- 3 Routes request to correct service(s)

- 4 Aggregates or transforms response
- 5 Sends response back to client

Simple diagram

Client

|

API Gateway

|----> User Service

|----> Order Service

|----> Payment Service

Common responsibilities

- Request routing
- Authentication & authorization
- Rate limiting
- Throttling
- DDos Protection
- SSL termination
- API composition
- Logging & monitoring
- Response Aggregation
- Response Caching
- Response Header/Body manipulation

Real-world examples

Example 1: Mobile App Backend

- Mobile app calls one API
- Gateway routes to:
 - User service
 - Order service
 - Notification service

Mobile app stays simple.

Example 2: Netflix-style Architecture

- Gateway tailored per client (Web, Mobile, TV)
- Each gateway optimizes responses for that client

Types of API Gateways

1 Single Gateway

- One gateway for all clients
- Simple but can become complex

2 Backend for Frontend (BFF)

- One gateway per client type

- Web, Mobile, Partner APIs

Benefits

- ☒ Single entry point
- ☒ Centralized security
- ☒ Reduced client complexity
- ☒ Better observability

Trade-offs

- ☒ Potential single point of failure
- ☒ Added latency
- ☒ Can become a monolith if not designed well

API Gateway vs API Composition

API Gateway

Entry point

Handles cross-cutting concerns

Often **used together**.

API Composition

Aggregation logic

Focuses on data aggregation

When should you use it?

Use it when:

- You have multiple microservices
- Multiple client types exist
- You need centralized security

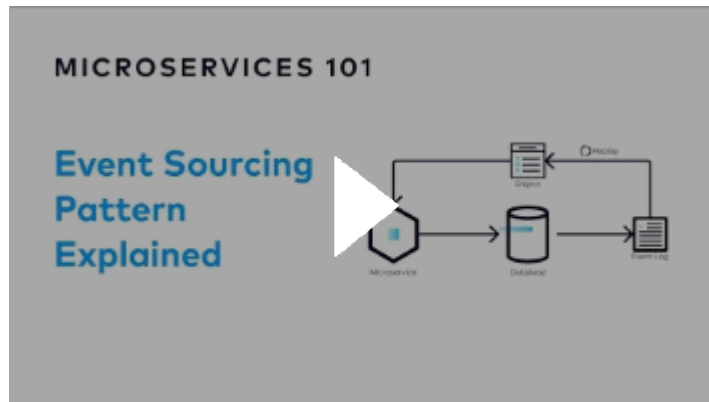
Avoid it when:

- System is very small
- Single service handles everything

Event Sourcing Pattern

22 January 2026 13:42

[What is the Event Sourcing Pattern? | Designing Event-Driven Microservices](#)



What is Event Sourcing?

Event Sourcing stores state as a sequence of immutable events and reconstructs current state by replaying them.

What problem does it solve?

Event Sourcing solves the problem of **losing history and state ambiguity**. Instead of storing only the **current state**, it stores **every state change as an event**.

What is Event Sourcing?

Event Sourcing stores an application's state as a sequence of immutable events. The current state is **reconstructed** by replaying these events.

Traditional vs Event Sourcing

Traditional (CRUD)

- ✗ No history
- ✗ Cannot answer "how did we get here?"

Event Sourcing

- ✓ Full history
- ✓ Time travel
- ✓ Auditability

How it works (step-by-step)

1 Command

A user performs an action (e.g., *Place Order*).

2 Event is created

If valid, an event is recorded : OrderPlaced

3 Event is stored

Events are appended to an **event store** (immutable).

4 State is rebuilt

Current state is calculated by replaying events.

Real-world example

Example: Bank Account

Instead of:

balance = 500

Store:

AccountOpened

MoneyDeposited(1000)

MoneyWithdrawn(500)

Current balance = replay events.

Example: E-commerce Order

Events:

OrderCreated

OrderConfirmed

OrderCancelled

You can:

- Audit changes
- Replay for debugging
- Rebuild projections

Event Store

- Append-only
- Events are immutable
- Acts as the **source of truth**

Common storage:

- Kafka
- EventStoreDB
- Database table

Read Models (Projections)

Since replaying events every time is slow:

- Build **read-optimized views**
- Update them asynchronously

This is often combined with **CQRS**.

Benefits

- ☒ Complete audit trail
- ☒ Easy debugging & time travel
- ☒ Natural fit for event-driven systems
- ☒ Supports complex business workflows

Trade-offs

- ☒ Higher complexity
- ☒ Event versioning required
- ☒ Harder learning curve
- ☒ Event schema evolution needed

When should you use Event Sourcing?

Use it when:

- Auditability is critical
- Business logic is complex
- History matters
- Event replay is valuable

Avoid it for:

- Simple CRUD systems
- Teams new to distributed systems

Strangler Fig Pattern

22 January 2026 13:42

[Master Microservices Strangler Pattern by Top AWS Experts](#)



The Strangler Fig Pattern incrementally replaces a legacy system by routing functionality to new implementations until the old system can be retired.

What problem does it solve?

The **Strangler Fig Pattern** solves the problem of **modernizing or replacing a large legacy system without a risky big-bang rewrite**. It allows you to **incrementally replace a monolith** with new services.

What is the Strangler Fig Pattern?

Gradually build a new system around an existing one, route traffic to the new parts, and eventually remove the old system. Named after the strangler fig tree, which grows around a tree and slowly replaces it.

How it works (step-by-step)

1 Identify a slice of functionality

Pick a small, well-defined feature (e.g., payments, reporting).

2 Build a new implementation

Implement this feature as:

- A microservice
- A new module
- A new API

3 Route traffic

Use:

- API Gateway
- Reverse proxy
- Load balancer

Route **only that functionality** to the new system.

4 Repeat

Gradually move more features.

5 Retire the legacy system

Once all functionality is replaced, decommission the monolith.

Simple flow diagram

Client

|

API Gateway

|-----> Legacy System

|

|-----> New Microservice

Over time:

Legacy → Smaller → Gone

Real-world examples

Example 1: E-commerce Monolith

Legacy app handles:

- Orders
- Payments
- Inventory

Step-by-step:

- Extract **Payments** as a microservice
- Route payment requests to new service
- Later extract Orders, then Inventory

Example 2: Bank Core System

- Legacy COBOL system
- New REST services built for:
 - Customer profiles
 - Notifications
- Gateway routes modern traffic to new services
- Legacy remains only for core functions until replaced

Why not rewrite everything at once?

- ✗ High risk
- ✗ Long timelines
- ✗ Hard to rollback

Strangler pattern provides:

- ☒ Incremental delivery
- ☒ Lower risk
- ☒ Continuous validation

Benefits

- ☒ Safe modernization
- ☒ Minimal downtime
- ☒ Gradual learning
- ☒ Business continuity

Trade-offs

- ✗ Temporary complexity
- ✗ Dual systems to maintain
- ✗ Requires good routing strategy

When should you use it?

Use it when:

- You have a large legacy system
- You want to move to microservices
- Downtime is unacceptable
- Business must continue during migration

Database Per Service Pattern

22 January 2026 13:42

[Ultimate Guide to Database Per Service pattern in Microservices](#)



What is Database Per Service?

Database per Service ensures that each microservice owns its data, preventing tight coupling and enabling independent evolution.

Database per Service can have

- All services access **different tables from same database**, but those tables do not depend directly on each other
- All services can have tables in **different schemas**, one schema is not visible to other services.
- All services have their **separate database altogether**.

What problem does it solve?

The **Database per Service pattern** solves the problem of **tight coupling between services caused by shared databases**. Each microservice owns and controls its **own data**.

What is Database per Service?

Every microservice has its own dedicated database (or schema) that no other service can access directly. All data access happens **only through the service's API**.

Why is this important?

In microservices:

- Data = ownership
- Shared DB = hidden coupling

This pattern enforces **true service autonomy**.

How it works

- Service A → Database A
- Service B → Database B
- No direct cross-service DB access
- Communication happens via **APIs or events**

Simple diagram

Order Service → Order DB

Payment Service → Payment DB

Inventory Service → Inventory DB

Real-world example

Example: E-commerce system

Service

Database

Order Service Orders DB

Payment Service Payments DB

Inventory Service Inventory DB

- Order Service cannot query Payment DB
- Payment updates are shared via **events**

Why shared database is an anti-pattern

- ✗ Tight coupling
- ✗ Schema changes break multiple services
- ✗ Hard to scale independently
- ✗ Difficult deployments

How services share data (without sharing DB)

1 API Calls

- Service calls another service for required data

2 Event-Driven

- Services publish events
- Other services build local copies (eventual consistency)

Data consistency model

- No distributed transactions
- **Eventual consistency** is preferred
- Often combined with:
 - Saga Pattern
 - Outbox Pattern

Benefits

- ☒ Independent scaling
- ☒ Independent deployments
- ☒ Technology flexibility (SQL, NoSQL, etc.)
- ☒ Clear ownership boundaries

Trade-offs

- ☒ Data duplication
- ☒ Complex consistency management
- ☒ Harder cross-service queries

Database choice flexibility

Each service can choose:

- Relational DB
- NoSQL
- Search engine
- Time-series DB

Based on its own needs.

When should you use it?

Use it when:

- Building true microservices
- Independent scaling is required
- Teams own services end-to-end

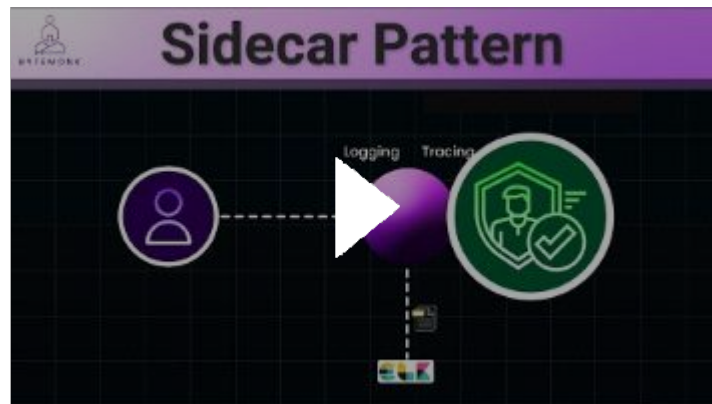
Avoid it when:

- System is small
- Strong consistency across entities is required

Sidecar Pattern

22 January 2026 13:42

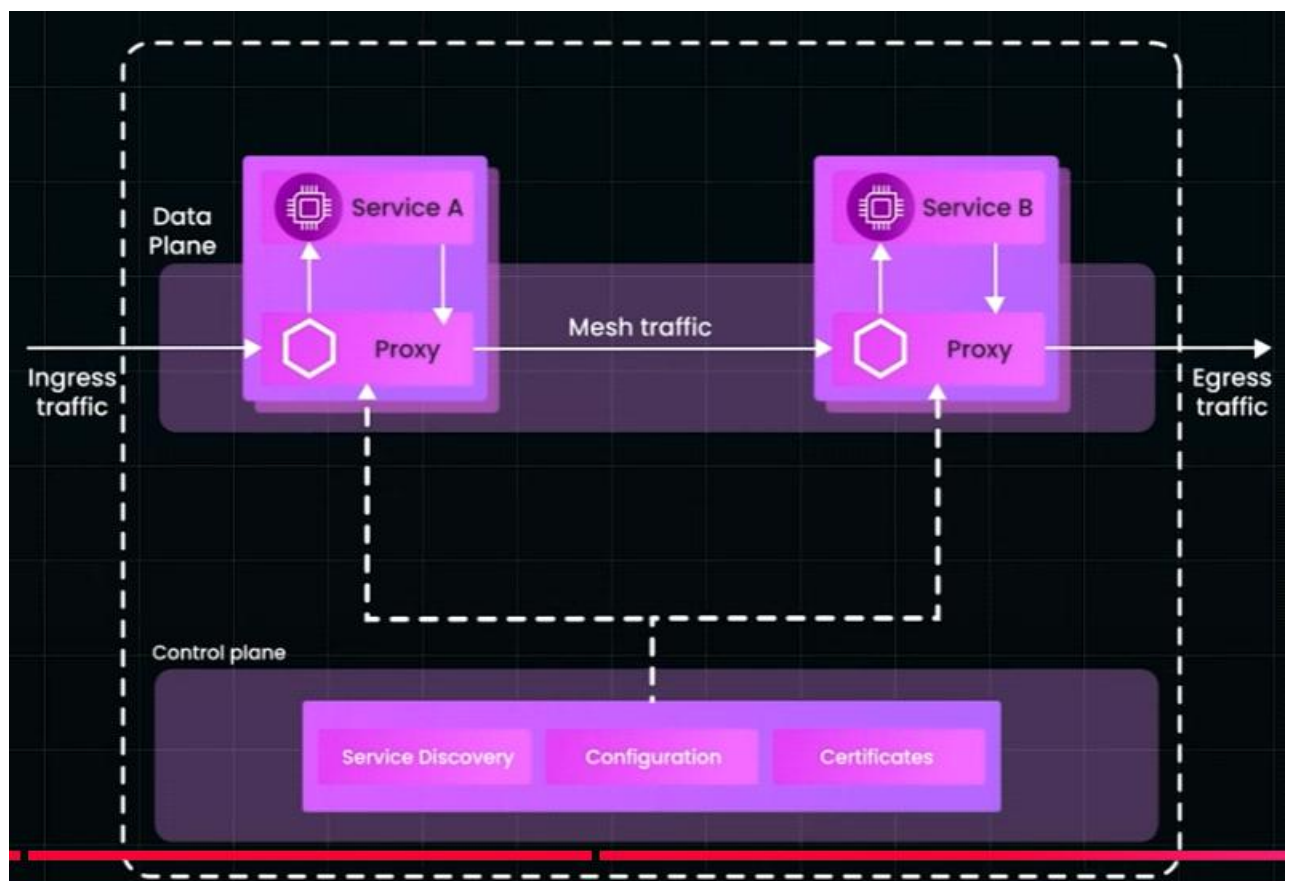
Sidecar Pattern in Microservices



What is Sidecar Pattern?

The Sidecar Pattern runs a helper component alongside a service to handle cross-cutting concerns without modifying application code.

The sidecar pattern is a core part of **Service Mesh Architecture**. A service mesh is a dedicated infrastructure layer for handling communication among microservices



Imagine serviceA needs to communicate to Service B in a service mesh,each service has its own sidecar proxy.

- ServiceA sends a request to Service B, the request first goes to service A sidecar proxy. The side car checks the routing rules,apply security policies and then forwards the request to service B sidecar proxy. Service B sidecar intercepts the request, checks for any security rules like authentication and then passes the request to service B. The response from service B follows the same path back followed through both side cars.

What problem does it solve?

The **Sidecar Pattern** solves the problem of **adding cross-cutting concerns** (logging, monitoring, security, config, networking) **without changing application code**. It keeps business logic clean and infrastructure concerns reusable.

What is the Sidecar Pattern?

A **sidecar** is a helper component that runs **alongside a service**, sharing the same lifecycle, and provides supporting functionality.

Think of it as an **assistant container/process** for your service.

How it works

- Application runs normally
- Sidecar runs next to it
- Sidecar handles infrastructure concerns
- App communicates with sidecar via:
 - Local network
 - Shared filesystem
 - IPC

Simple diagram

[Application]

|

[Sidecar]

In containers:

Pod

├─ App Container

└─ Sidecar Container

Real-world examples

Example 1: Service Mesh (Very common)

- Envoy sidecar injected per service
- Handles:
 - Traffic routing
 - Retries
 - mTLS
 - Observability

Used in Istio, Linkerd

Example 2: Logging Agent

- App writes logs to stdout/file
- Sidecar collects logs
- Ships them to ELK / Splunk

Example 3: Configuration / Secrets

- Sidecar fetches secrets from Vault
- App reads them locally
- App never talks to Vault directly

Why not put everything inside the app?

- ✗ Duplicated code
- ✗ Hard to update infra logic
- ✗ Tight coupling

Sidecar provides:

- ☒ Reuse
- ☒ Separation of concerns
- ☒ Independent upgrades

Benefits

- ☒ Keeps application code simple
- ☒ Consistent behavior across services
- ☒ Technology-agnostic
- ☒ Easy to add/remove features

Trade-offs

- ✗ Extra resource usage
- ✗ Increased operational complexity
- ✗ More moving parts
- ✗ Adds up latency

Sidecar vs Library approach

Sidecar	Library
Language-agnostic	Language-specific
Separate lifecycle	Tied to app
Central updates	App rebuild needed

When should you use it?

Use it when:

- You need common infrastructure logic
- Multiple services need same behavior
- You want zero code changes in apps

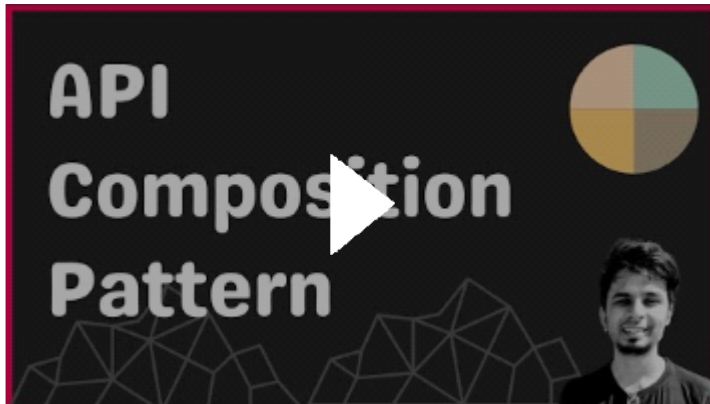
Avoid it when:

- App is very simple
- Resource constraints are strict

Api Composition Pattern

22 January 2026 13:45

[API Composition Pattern in Microservices](#)



What is API Composition?

API Composition aggregates data from multiple microservices into a single API response, simplifying client interactions.

What problem does it solve?

The **API Composition Pattern** solves the problem of **clients needing data from multiple microservices**. Instead of the client calling many services, a **composite API** gathers and returns the data in one call.

What is API Composition?

A service (or layer) that calls multiple backend services, combines their responses, and returns a single response to the client.

The client sees **one API**, not many.

Why is this needed?

Without API composition:

- Clients make multiple network calls
- Client logic becomes complex
- Tight coupling to service structure

API composition:

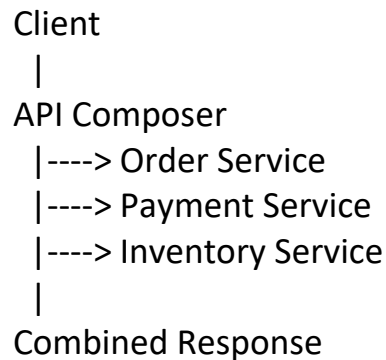
- ☒ Simplifies clients
- ☒ Hides microservice complexity

How it works

- 1 Client calls Composite API
- 2 Composite API calls multiple services

- 3 Aggregates results
- 4 Returns a unified response

Simple diagram



Real-world examples

Example 1: E-commerce Order Details

Client needs:

- Order info
- Payment status
- Shipping details

API Composition:

- Order API → Order Service
- Payment API → Payment Service
- Shipping API → Shipping Service

Client gets **one response**.

Example 2: User Profile Page

User profile shows:

- User info
- Recent orders
- Notifications

Composite API fetches data from:

- User Service
- Order Service
- Notification Service

Where does API Composition live?

- Dedicated Composition Service
- API Gateway
- Backend-for-Frontend (BFF)

API Composition vs Client-Side Composition

API Composition	Client Composition
Server-side	Client-side

Fewer network calls Many calls
Better control More client complexity

Benefits

- ☒ Simple client
- ☒ Reduced network chatter
- ☒ Better abstraction
- ☒ Centralized orchestration

Trade-offs

- ☒ Extra hop
- ☒ Potential performance bottleneck
- ☒ Partial failure handling required

Failure handling

Composite API must handle:

- Timeouts
- Partial responses
- Fallbacks

Often combined with:

- Circuit Breaker
- Timeout patterns

API Composition vs CQRS

API Composition

Aggregates APIs

Orchestration pattern

CQRS

Separates reads/writes

Architectural pattern

They can be **used together**.

When should you use it?

Use it when:

- UI needs data from multiple services
- You want a clean client contract
- Latency can be optimized server-side

Avoid it when:

- Single service can serve the request
- Extreme low-latency is required

Feign Client Pattern

22 January 2026 13:42

A **Feign Client** is a **declarative HTTP client** used in Spring Cloud to make REST calls between microservices *without writing boilerplate code*.

Instead of manually using **RestTemplate** or **WebClient**, you simply create a **Java interface**, add annotations, and Feign generates the implementation.

☑ Simple Definition

A **Feign Client** lets one microservice call another microservice's REST API **as if it were a normal Java method**, with zero manual HTTP handling.

🤖 Why Feign is used

Because it:

- ✓ Removes boilerplate REST call code
- ✓ Supports load-balancing (with Eureka)
- ✓ Integrates with Spring Cloud (Resilience4j, Ribbon, etc.)
- ✓ Makes microservice-to-microservice communication clean

🔗 How Feign Works (In One Sentence)

You define an interface → Feign auto-generates an HTTP client behind the scenes
→ Calls remote services like normal methods.

🔗 Example Feign Client (Very Simple)

```
java

@FeignClient(name = "crew-service")
public interface CrewClient {

    @GetMapping("/api/crew/{id}")
    CrewDto getCrewById(@PathVariable("id") Long id);
}
```

📋 Copy code

Usage:

```
java                                                                    Copy code

@Service
public class SchedulingService {

    @Autowired
    private CrewClient crewClient;

    public CrewDto fetchCrew(Long id) {
        return crewClient.getCrewById(id); // REST call, but feels like a Java method
    }
}
```

What Feign Does Automatically

- Creates HTTP request for you
- Serializes / deserializes JSON
- Handles URL building
- Integrates with load balancers
- Adds headers (JWT, tracing, etc.)
- Supports fallback with Resilience4j

In an Airline Microservices System

Examples where Feign Client is ideal:

- **Scheduling Service → Crew Service**
- **Scheduling Service → Fleet Service**
- **Auth Service → User Profile Service**
- **Reporting Service → Flight Service**

For example:

"Give me the latest aircraft status" or "Check if a pilot is available."

Feign makes these calls as simple as calling a local method.

In short

Feign Client = Simplified microservice-to-microservice REST communication.

Dual Write Problem

22 January 2026 13:55

What is the Dual Write Problem?

The **dual write problem** happens when an application must write the **same business change to two different systems** (usually a database + something else) and **cannot guarantee atomicity across them**.

Typical examples

- Write to **DB + Kafka**
- Write to **DB + Cache**
- Write to **DB + Search index (Elasticsearch)**
- Write to **DB + another DB / microservice**

Why it's a problem

If one write succeeds and the other fails, your system becomes **inconsistent**.

Example

1. Save Order to DB ☒
2. Publish OrderCreated event to Kafka **✗**

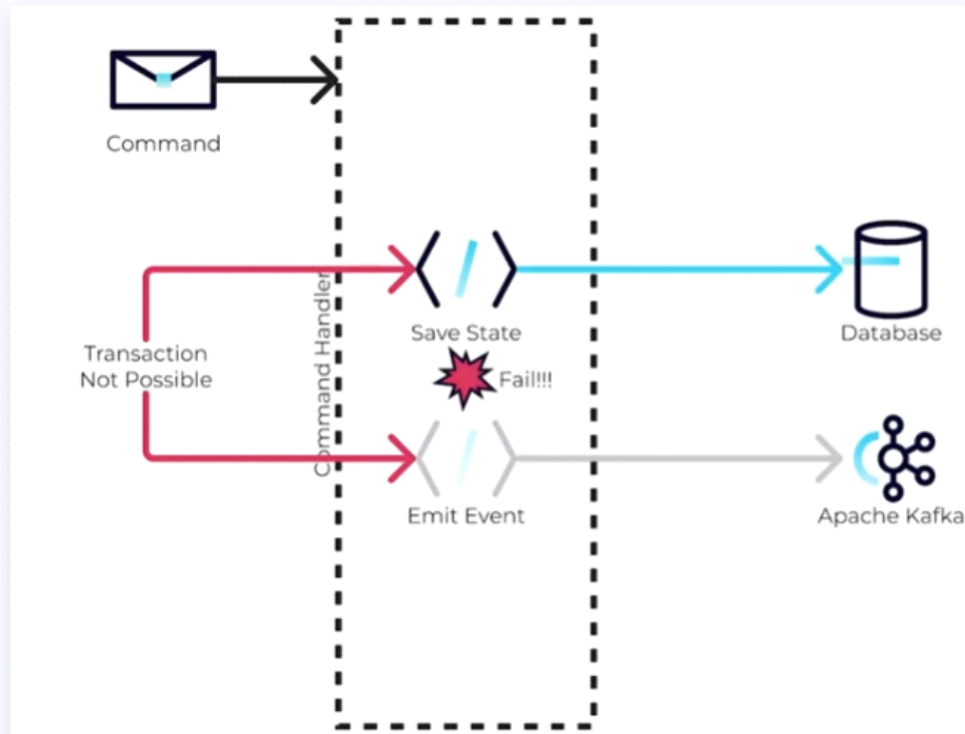
Now:

- DB says the order exists
- Downstream systems never hear about it

This breaks:

- Data consistency
- Event-driven flows
- Auditability
- Recovery logic

The Dual Write Problem



The database is updated, but the event never gets emitted.

Why It's Hard

- Distributed systems **do not support atomic transactions** across heterogeneous systems
- Network failures, retries, timeouts are normal
- Two-phase commit (2PC) is **slow, fragile, and rarely used** at scale

Proven Solutions (Best → Worst)

☒ 1. Transactional Outbox Pattern (Best Practice)

Idea:

Write business data and events **together in the same DB transaction**.

How it works

- Write domain data → orders table
- Write event → outbox_events table
- Commit once (atomic)
- A separate process publishes events to Kafka

DB Transaction:

- INSERT order
- INSERT outbox_event
COMMIT

Why it works

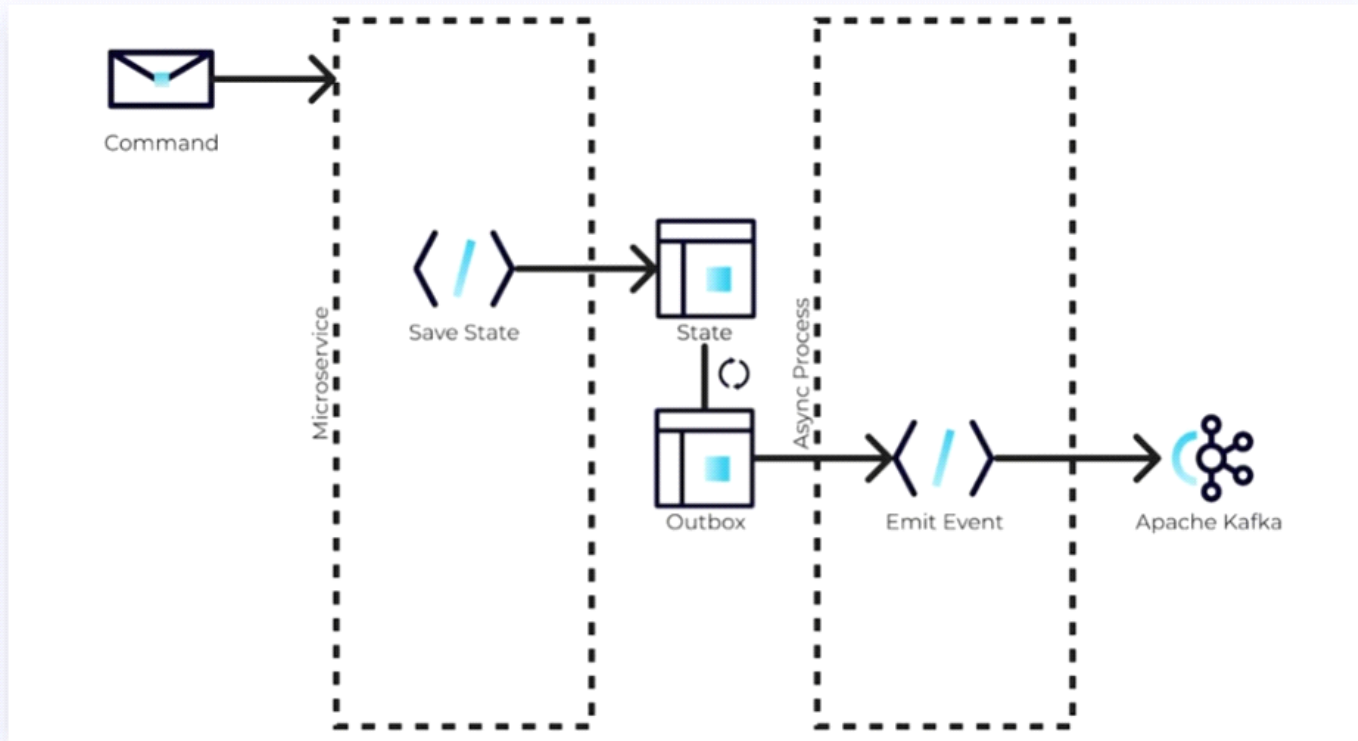
- Single atomic transaction
- Guaranteed consistency

- Replayable & auditable

Used by

Netflix, Uber, AWS, most serious event-driven systems

Patterns: Transactional Outbox



The Transactional Outbox uses a separate Outbox table to record outgoing events.

✓ 2. Change Data Capture (CDC)

Idea:

Let the database tell you what changed.

How it works

- App writes **only to DB**
- CDC tool (Debezium, AWS DMS) reads DB logs
- Changes are streamed to Kafka

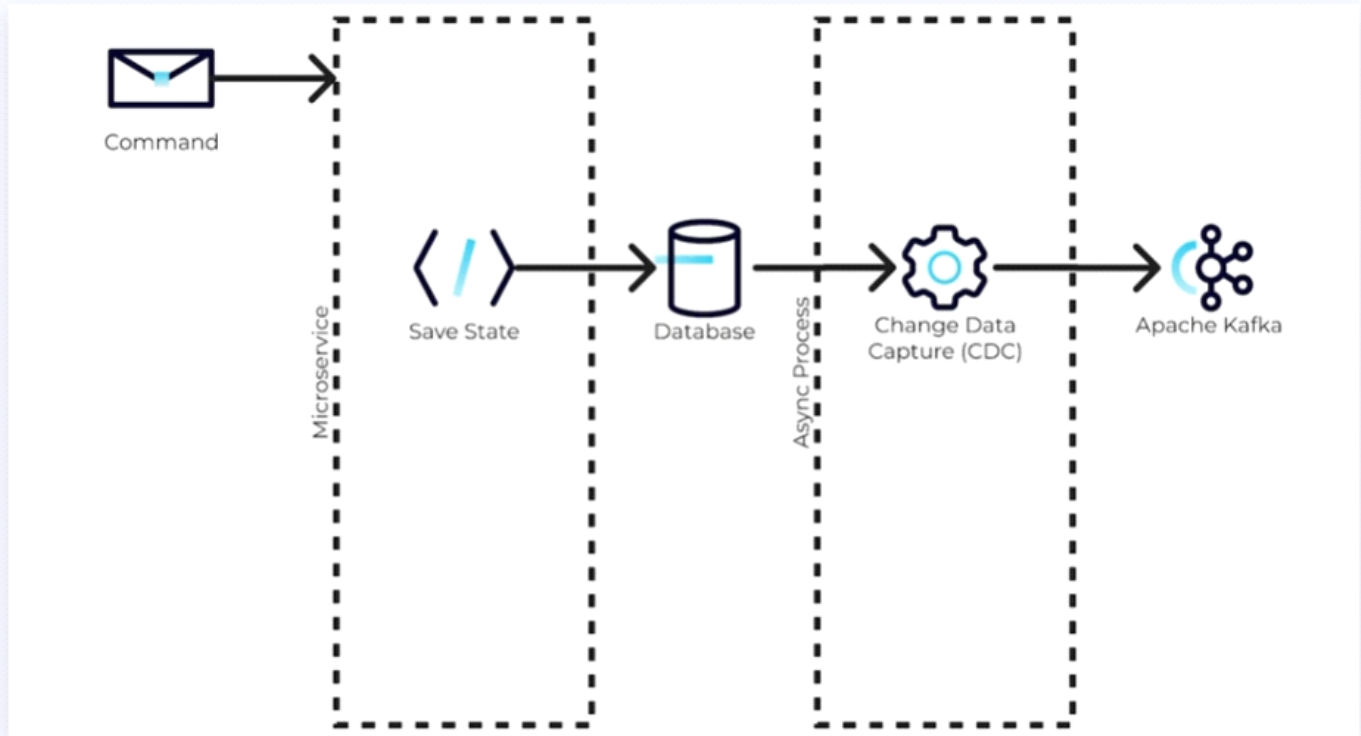
Pros

- No dual writes in application code
- Extremely reliable
- Scales well

Cons

- Infra complexity
- Event format tied to DB schema

Patterns: Change Data Capture (CDC)



Change Data Capture (CDC) watches for database changes and can emit events.

☑ 3. Idempotent Writes + Retry (Mitigation, not a fix)

Idea:

Accept failure but make retries safe.

How it works

- Use idempotency keys
- Retry failed writes safely
- Detect duplicates

Why it's weak

- Still inconsistent during failures
- Complex recovery logic
- Hard to reason about correctness

Use this **only** as a complement to Outbox/CDC.

⚠ 4. Distributed Transactions (2PC)

Idea:

Try to commit both systems together.

Why it's bad

- Slow
- Locks resources
- Poor failure handling
- Doesn't scale

Almost always rejected in modern microservices.

⚠ 5. Best-Effort Dual Write (Anti-pattern)

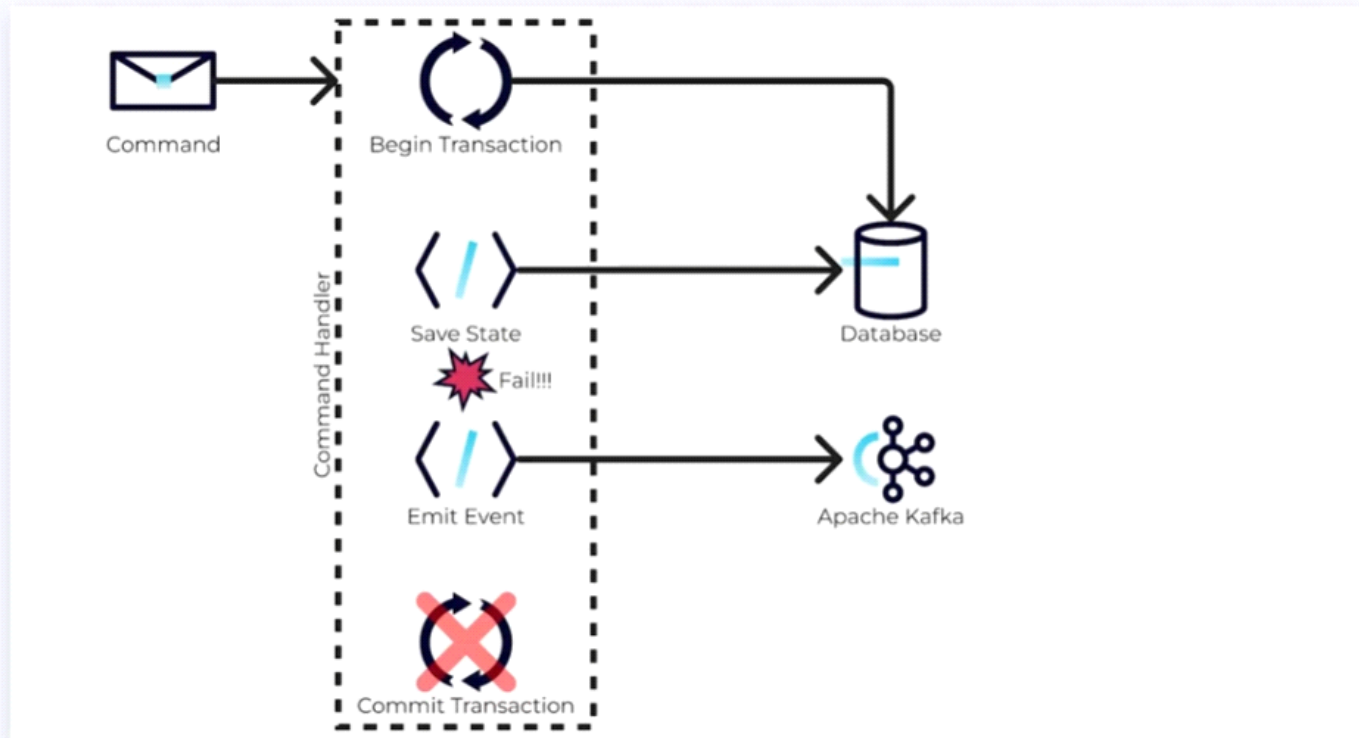
```
try {
```

```
writeDB()  
writeKafka()  
} catch {}
```

✗ Eventually leads to:

- Silent data loss
- Broken downstream systems
- Nightmares in production

Invalid Solution: Use a Transaction



If a failure occurs, then the transaction gets rolled back.

The dual write problem occurs when a service must update multiple systems without atomicity, leading to inconsistency. The preferred solution is the Transactional Outbox or CDC to guarantee reliable event publication.

12 Factor App

22 January 2026 18:01

The 12 Factors of Microservice Architecture (12-Factor App)

1 Codebase

One codebase per application, tracked in version control. A 12 factor app should be tracked in a single code repository and must not share that repository with any other apps.

🔗 *Example:* One Git repo deployed to dev, stage, and prod.

2 Dependencies

Explicitly declare all dependencies. The twelve-factor app should always explicitly declare all its dependencies

🔗 *Example:* Using package.json or pom.xml instead of system-installed libraries.

3 Config

Store configuration in environment variables. A twelve-factor app should externalize all such configurations that vary between deployments

🔗 *Example:* DB_URL set via env variable, not hardcoded.

4 Backing Services

Treat external services as attached resources. A twelve-factor app should treat all such backing services as attached resources

🔗 *Example:* Switching from local DB to cloud DB via config only.

5 Build, Release, Run

Strictly separate build and runtime stages. The twelve-factor methodology strictly separates the process of converting codebase into a running application

🔗 *Example:* Build Docker image → inject config → run container.

6 Processes

Run the app as stateless processes. A twelve-factor app is expected to run in an execution environment as stateless processes

🔗 *Example:* User sessions stored in Redis, not in memory.

7 Port Binding

Expose services via a port. A twelve-factor app, on the contrary, expects no such runtime dependency

🔗 *Example:* Spring Boot app running on port 8080.

8 Concurrency

Scale by running multiple instances. The twelve-factor methodology suggests apps to rely on processes for scaling. However, free to leverage a concurrency model like *Thread* internally

☞ *Example:* Run 5 containers instead of adding threads.

9 Disposability

Fast startup and graceful shutdown. Application processes can be shut down on purpose or through an unexpected event. In either case, **a twelve-factor app is supposed to handle it gracefully**

☞ *Example:* App shuts down cleanly during Kubernetes scaling.

10 Dev/Prod Parity

Keep dev, staging, and prod environments similar. The twelve-factor methodology suggests keeping the gap between development and production environment as minimal as possible

☞ *Example:* Same database type across all environments.

1 1 Logs

Treat logs as event streams. A twelve-factor app, however, separates itself from log generation and its processing. **For such an app, logs are nothing but a time-ordered stream of events.**

☞ *Example:* Log to stdout, let ELK or CloudWatch collect them.

1 2 Admin Processes

Run admin tasks as one-off processes. The twelve-factor methodology strongly suggests keeping such admin scripts together with the application codebase

☞ *Example:* Database migration run as a separate command.

Deployment Strategies

23 January 2026 11:49

Deployment Strategies

1 Recreate Deployment

◇ What It Is

- Old version is completely stopped.
- New version is deployed after shutdown.
- No overlap between versions.

◇ How It Works

1. Stop all existing instances.
2. Deploy new version.
3. Start new instances.

◇ Pros

- Very simple to implement.
- No version compatibility issues.
- No double infrastructure cost.

◇ Cons

- Downtime is unavoidable.
- Risky if deployment fails.
- Poor user experience for high-traffic systems.

◇ When to Use

- Internal tools.
- Low-traffic apps.
- Non-critical batch systems.
- Schema-breaking database changes (sometimes required).

◇ Interview Insight

If you mention this, say:

"I avoid recreate in customer-facing production unless downtime is acceptable."

2 Rolling Deployment (Rolling Update)

◇ What It Is

- Gradually replaces old instances with new ones.
- Ensures some capacity is always serving traffic.

◇ How It Works

- Replace instances in batches (e.g., 2 at a time).
- Health checks validate new instances.
- Continue until all old instances are replaced.

In Kubernetes, controlled by:

- maxUnavailable
- maxSurge

◇ Pros

- No full downtime.

- Infrastructure efficient.
- Gradual rollout reduces risk.

◇ Cons

- Old and new versions coexist → backward compatibility required.
- Rollback takes time.
- Database migrations must be carefully managed.

◇ When to Use

- Standard web applications.
- Microservices.
- Moderate-risk changes.

◇ Interview Insight

“I ensure backward compatibility before rolling updates to avoid runtime failures.”

3 Blue-Green Deployment

◇ What It Is

Two identical environments:

- Blue → Current live version
- Green → New version

Traffic is switched after validation.

◇ How It Works

1. Deploy new version to Green.
2. Run smoke tests.
3. Switch load balancer to Green.
4. Keep Blue as rollback option.

◇ Pros

- Instant rollback.
- No downtime.
- Full validation before traffic shift.

◇ Cons

- Requires double infrastructure.
- Database schema changes must be backward compatible.
- Slightly higher cost.

◇ When to Use

- High availability systems.
- Customer-facing platforms.
- Enterprise SaaS.

◇ Interview Insight

“For critical services, I prefer blue-green because rollback is immediate and low risk.”

4 Canary Deployment

◇ What It Is

- Release to a small % of users (e.g., 5%).
- Gradually increase exposure.

◇ How It Works

1. Deploy new version.
2. Route small % of traffic.

3. Monitor metrics (error rate, latency, CPU).
4. Increase gradually if stable.

◇ Pros

- Real-world validation.
- Reduces blast radius.
- Data-driven confidence.

◇ Cons

- Requires strong monitoring.
- More complex routing.
- Harder debugging if user-specific issues arise.

◇ When to Use

- Risky features.
- Performance-sensitive systems.
- Machine learning models.
- Large-scale consumer platforms.

◇ Interview Insight

“I combine canary with automated rollback thresholds to avoid manual delay.”

5 A/B Testing Deployment

◇ What It Is

- Different user groups see different versions.
- Used to measure business impact.

◇ Difference from Canary

- Canary → Stability validation.
- A/B Testing → Business experimentation.

◇ Pros

- Data-driven product decisions.
- Improves conversion metrics.
- Validates UX changes.

◇ Cons

- Requires analytics maturity.
- May increase code complexity.
- Needs experiment governance.

◇ When to Use

- UI/UX changes.
- Pricing experiments.
- Recommendation systems.

◇ Interview Insight

“I ensure experiments are time-bound and statistically significant before rollout.”

6 Shadow Deployment (Dark Launch)

◇ What It Is

- New version receives real production traffic.
- Does NOT serve responses to users.
- Used for testing performance & scalability.

◇ How It Works

- Mirror traffic from production.
- Compare responses/logs.
- Measure performance impact.

◇ Pros

- Tests production load safely.
- Detects performance bottlenecks.
- Zero user risk.

◇ Cons

- Requires extra infrastructure.
- Complex observability needed.

◇ When to Use

- High-scale system upgrades.
- Infrastructure rewrites.
- Database engine changes.

◇ Interview Insight

“We used shadow traffic before migrating a core service to ensure latency didn’t regress.”

7 Feature Flag (Toggle-Based Deployment)

◇ What It Is

- Code deployed but feature hidden behind configuration flag.
- Controlled activation.

◇ How It Works

- Deploy code safely.
- Turn on feature for:
 - Internal users
 - Beta users
 - % rollout

◇ Pros

- Reduces deployment risk.
- Instant rollback (disable flag).
- Enables gradual exposure.
- Decouples deployment from release.

◇ Cons

- Flag debt if not cleaned up.
- Increased code complexity.
- Requires governance.

◇ When to Use

- Most modern SaaS systems.
- Continuous delivery pipelines.

◇ Interview Insight

“I treat feature flags as temporary infrastructure and ensure cleanup is part of sprint hygiene.”



How to Choose Between Them (Interview Framing)

If interviewer asks:

“Which deployment strategy would you choose?”

Answer structure:

1. Assess risk level.
2. Evaluate user impact.
3. Check infrastructure capability.
4. Decide rollback requirement.

Example:

“For a high-risk change in a high-traffic system, I’d combine feature flags with canary deployment and automated rollback. For mission-critical systems requiring instant rollback, I’d prefer blue-green.”

Advanced Manager-Level Talking Points

Mention these to stand out:

- Automated rollback thresholds
- Observability (SLIs, SLOs)
- Deployment health checks
- Backward-compatible schema migrations
- Decoupling deployment from release
- Infrastructure cost trade-offs
- Blast radius reduction

10-Second Summary for Revision

- **Recreate → Simple, downtime.**
- **Rolling → Gradual replacement.**
- **Blue-Green → Two environments, instant rollback.**
- **Canary → Small % rollout.**
- **A/B → Business experiment.**
- **Shadow → Traffic mirroring.**
- **Feature Flag → Controlled activation.**

Resource

18 March 2026 16:53

<https://medium.com/%40dhruvtechbytes/deployment-strategies-explained-from-downtime-to-zero-risk-releases-b3cea169f6bf>

Deployment Strategies Explained: From Downtime to Zero-Risk Releases

Shipping code to production is easy...

Shipping it **without breaking production** is where real engineering begins.

If you've ever wondered how companies like Netflix or Amazon deploy multiple times a day without downtime, the answer lies in **deployment strategies**.

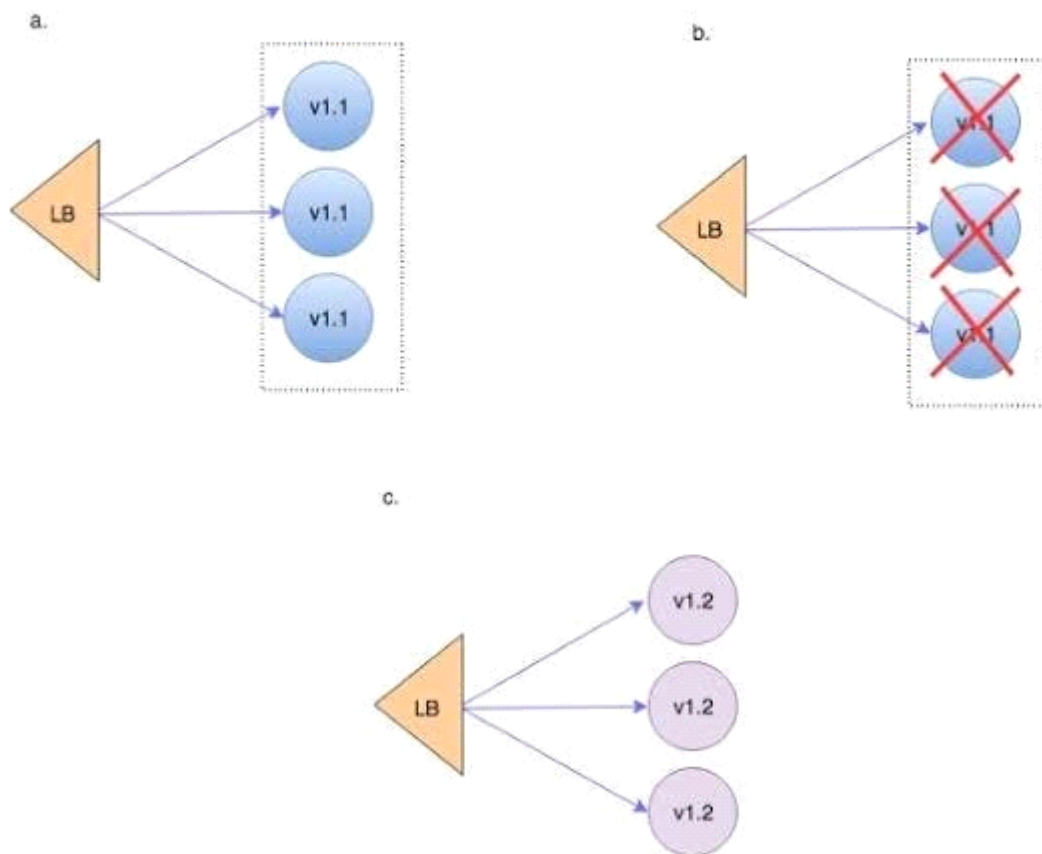
In this article, we'll break down the most important deployment patterns every engineer should know:

- Recreate
- Rolling
- Blue-Green
- Canary
- Feature Toggle
- Shadow Deployment

1. Recreate Deployment (The Simplest One)

This is the most basic deployment strategy.

Recreate



How it works:

You **stop the old version** and **deploy the new version**.

v1 **running** → stop v1 → deploy v2 → v2 **running**

Pros:

- Very simple to implement
- No complexity

Cons:

- **✗** Causes **downtime**
- Risky for production systems

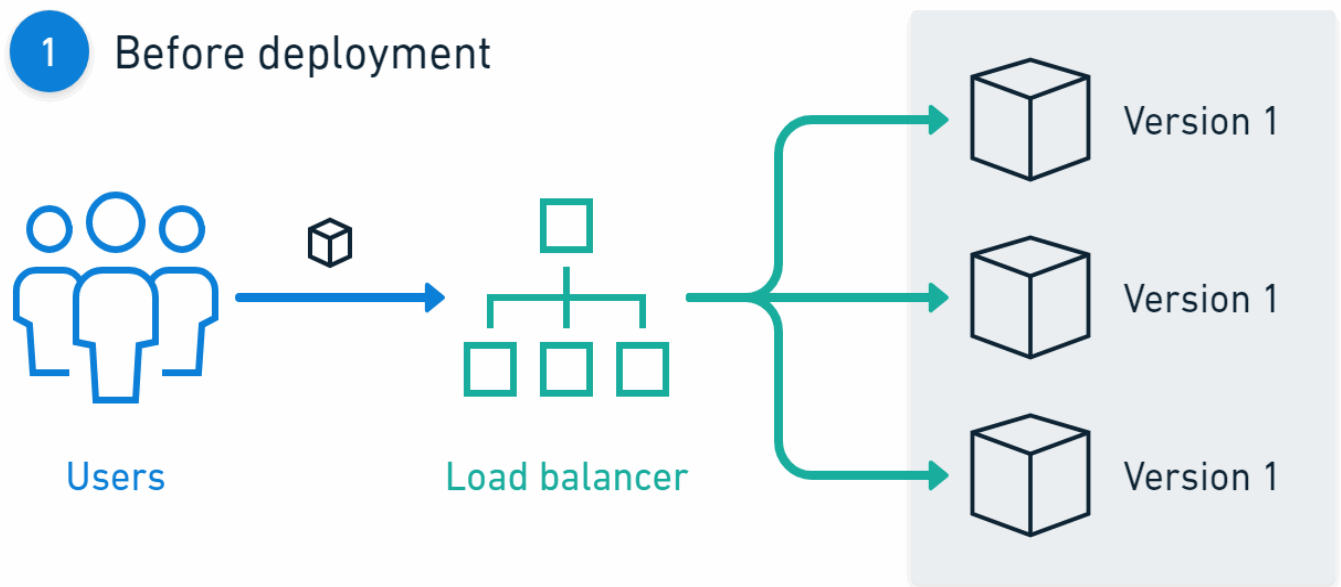
When to use:

- Internal tools
- Low-traffic applications

☞ Think of it as: “*Turn off → Update → Turn on*”

2. Rolling Deployment (Gradual Replacement)

Instead of replacing everything at once, we update **instances gradually**.



How it works:

v1 v1 v1 v1

↓

v2 v1 v1 v1

↓

v2 v2 v1 v1

↓

v2 v2 v2 v2

Pros:

- No downtime
- Resource efficient

Cons:

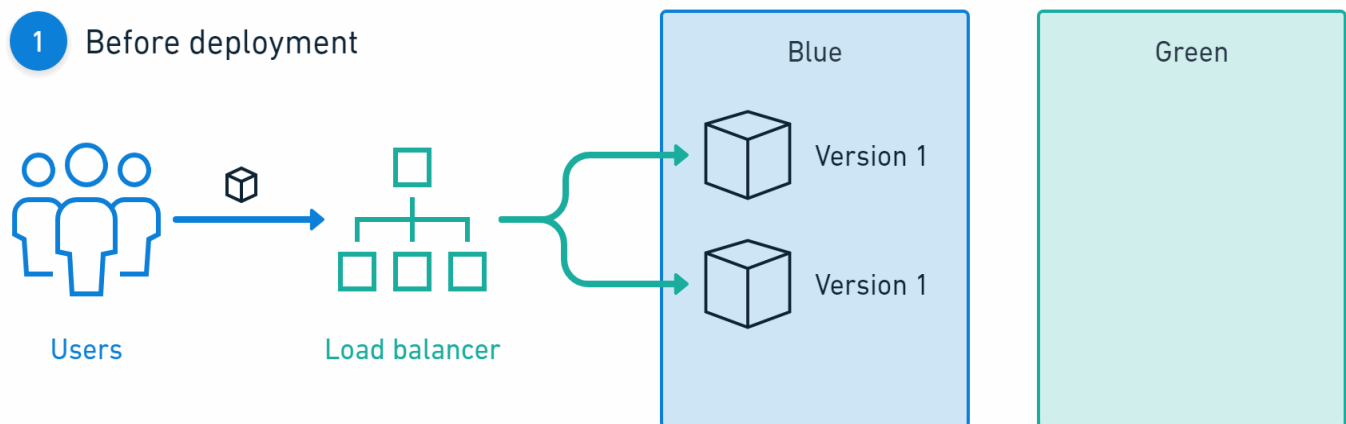
- Multiple versions run simultaneously
- Harder debugging & rollback

When to use:

- Microservices
- Kubernetes-based systems

👉 This is the **default strategy in Kubernetes**

3. Blue-Green Deployment (Zero Downtime Switch)



Here, we maintain **two identical environments**:

- **Blue** → **Current production**
- **Green** → **New version**

How it works:

1. Deploy new version to Green
2. Test it
3. Switch traffic from Blue → Green

Pros:

- Zero downtime
- Instant rollback

Cons:

- Requires **double infrastructure**

When to use:

- Critical production systems
- High availability applications

👉 Think: “Switch the entire system in one click”

4. Canary Deployment (Safe & Smart Rollout)

Instead of releasing to everyone, we release to a **small percentage of users first**.

How it works:

5% users → v2
95% users → v1

If everything works fine → gradually increase traffic.

Pros:

- Very low risk
- Detect issues early

Cons:

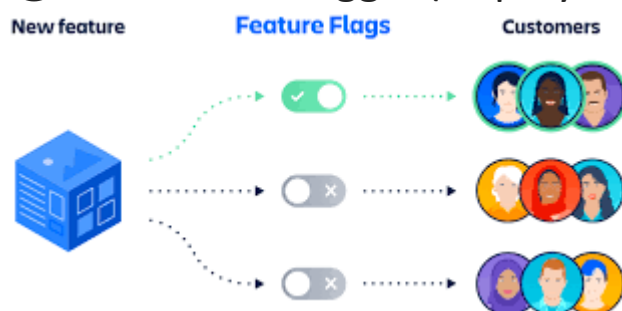
- Requires strong monitoring
- Slightly complex setup

When to use:

- Large-scale systems
- User-facing features

👉 This is how big tech avoids disasters in production.

5. Feature Toggle (Deploy ≠ Release)



In this strategy, you deploy code but **control visibility using flags**.

How it works:

```
if(featureFlagEnabled) {  
  newFeature();  
}
```

```
} else {  
oldFeature();  
}
```

Pros:

- Instant rollback (just turn off flag)
- Decouple deployment from release

Cons:

- Code complexity increases
- Requires flag management

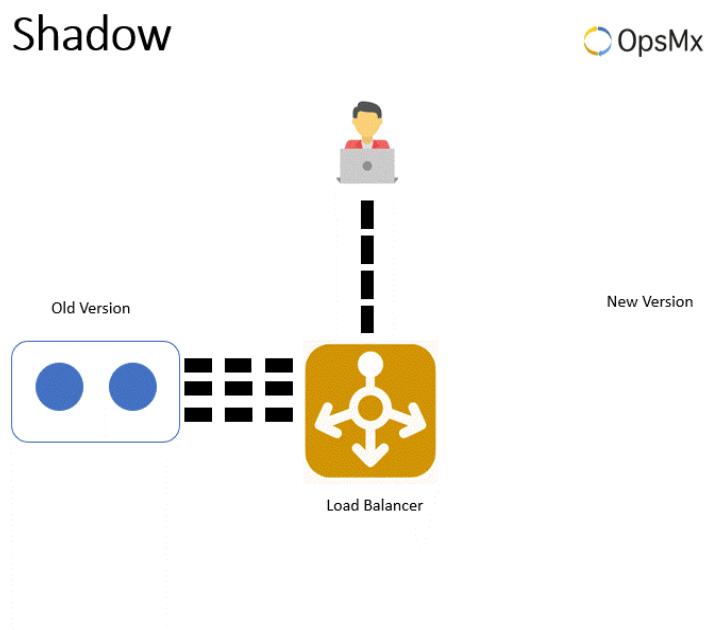
When to use:

- Continuous deployment
- Experimentation

👉 This is the backbone of **modern CI/CD pipelines**

● 6. Shadow Deployment (Dark Launch)

This is one of the most underrated strategies.



How it works:

- Real user traffic is **copied** to the new system
- Users still see response from old system

User → v1 (response shown)
→ v2 (tested silently)

Pros:

- Test with real production traffic
- No user impact

Cons:

- Requires extra infrastructure
- Complex to implement

When to use:

- Performance testing
- Backend migrations

👉 Think: *“Test in production without users knowing”*

⚖️ Quick Comparison

⚖️ Quick Comparison

Strategy	Downtime	Risk Level	Complexity
Recreate	❌ Yes	High	Low
Rolling	✅ No	Medium	Medium
Blue-Green	✅ No	Low	High
Canary	✅ No	Very Low	High
Feature Toggle	✅ No	Very Low	Medium
Shadow	✅ No	Very Low	High

🧠 Final Thoughts

In real-world systems, companies rarely use just one strategy.

They combine them:

- **Feature Flags + Canary** → **Controlled & safe rollout**
- **Blue-Green + Canary** → **Zero downtime + gradual exposure**

- **Rolling + Feature Flags** → Efficient + flexible

💡 Takeaway

If you're preparing for **system design interviews** or building production systems:

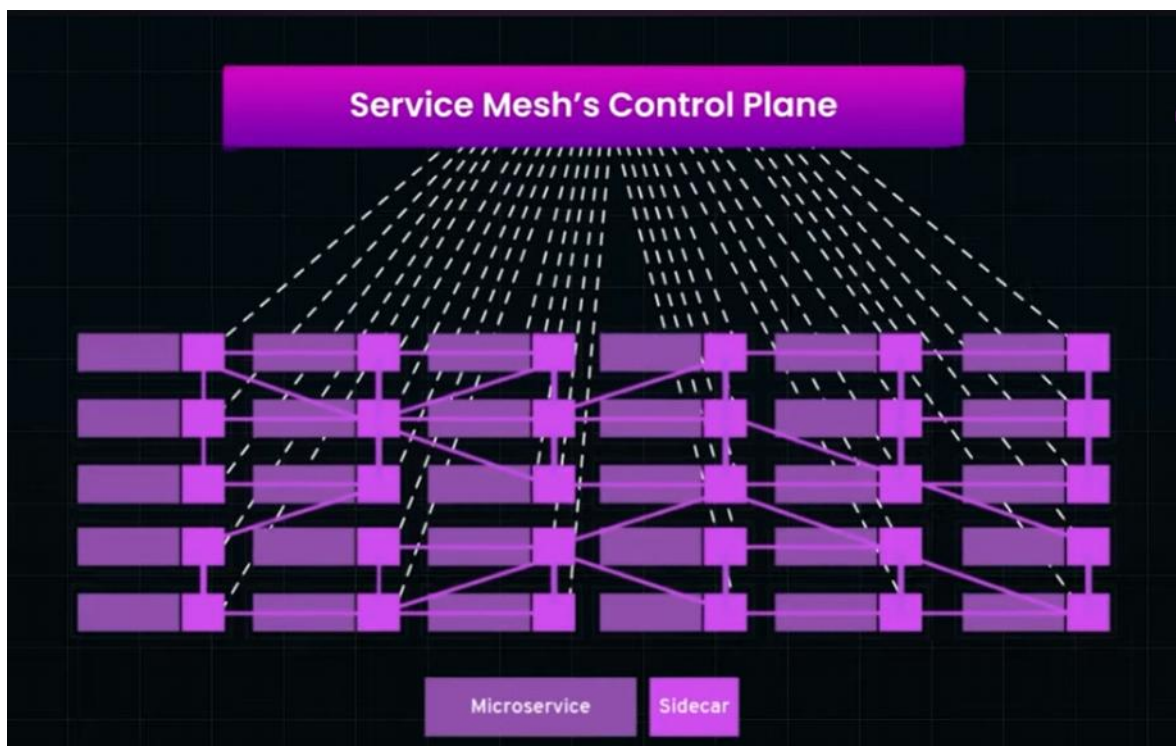
- Start simple → Rolling
- Move to safer → Canary
- Scale with → Feature Flags + Shadow

Because deployment is not just about shipping code...

👉 It's about **shipping safely, confidently, and repeatedly.**

Definition and Role of a Service Mesh

- A **service mesh** is an infrastructure layer dedicated to managing **service-to-service communication** in microservices architectures.
- It operates consists of a network of **sidecar proxies** deployed alongside each microservice, which handle all network traffic between services.
- Unlike traditional networking, a service mesh operates at the **application layer**, abstracting communication complexity.
- It enables developers to focus on code while the service mesh manages:
 - ▶ **Traffic management**
 - ▶ **Load balancing**
 - ▶ **Retries**
 - ▶ **Security**
 - ▶ **Observability**



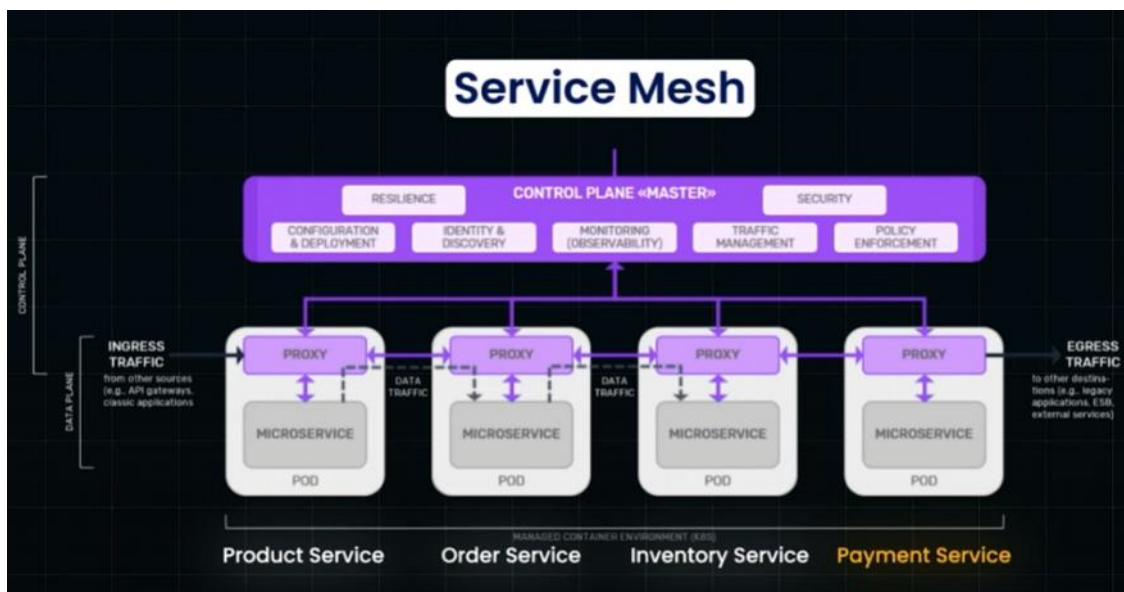
Challenges in Microservices Communication Without Service Mesh

- As the number of microservices grows (dozens to hundreds), managing inter-service communication becomes complex.
- Key challenges include:
 - Handling **network failures** (e.g., service downtime or slow responses)
 - Ensuring **secure communication** between services
 - **Monitoring and tracing** interactions across distributed systems

- Implementing **load balancing** for optimal performance
- Performing **zero-downtime deployments** of new service versions
- Manually handling these through custom code is inefficient and error-prone, highlighting the value of a service mesh as a **consistent, out-of-the-box solution**.

How Service Mesh Works via Sidecar Proxies

- Sidecar proxies are **deployed alongside each microservice** and intercept all network traffic.
- This enables the service mesh to manage communication **independently of the service code**.
- The video references a detailed explanation of sidecar proxies with examples (not included here).



Example: E-commerce Microservices Architecture

- Typical microservices in an e-commerce platform:
 - ▶ **Product service:** manages product details
 - ▶ **Order service:** handles customer orders
 - ▶ **Inventory service:** tracks stock levels
 - ▶ **Payment service:** processes transactions
- Without a service mesh, these services communicate directly, which can lead to issues if one service (e.g., payment) slows down or crashes.

Service Mesh Benefits Illustrated Through the E-commerce Example

- The sidecar proxy for each service handles retries when the payment service is unresponsive, preventing cascading failures.
- Traffic between services is **encrypted**, protecting sensitive information such as customer data.
- New versions of services (e.g., order service) can be rolled out via **Canary deployments**, where a small portion of traffic is routed to the new version before full rollout.
- This approach avoids embedding complex traffic management and security logic into service code.

Advanced Traffic Management Capabilities

- Service mesh enables **fine-grained traffic routing** based on:
 - Service versions
 - Load conditions
 - User-specific rules
- Example: Canary deployments for an inventory service can start with 10% of traffic to a new version and gradually increase as confidence grows.

Automatic Failure Handling and Traffic Resilience

- Failures in microservices are inevitable; service mesh helps by:
 - Automatically retrying failed requests based on predefined policies
 - Rerouting traffic to alternative services if needed
 - Gracefully degrading functionality to maintain availability

Security via Mutual TLS (mTLS)

- Service mesh employs **Mutual TLS (mTLS)** to ensure all communications between services are **secure and encrypted**.
- This eliminates the need to implement encryption manually within each microservice.
- Example: communication between order and payment services is encrypted by default, protecting sensitive data.

Observability Features

- A critical challenge in microservices is understanding real-time interactions and performance.
- Service mesh provides built-in **observability tools** such as:
 - **Distributed tracing**: tracks the journey of requests across multiple services
 - **Metrics collection**: monitors service health and performance
- These tools help detect bottlenecks, failures, and anomalies within the distributed system.

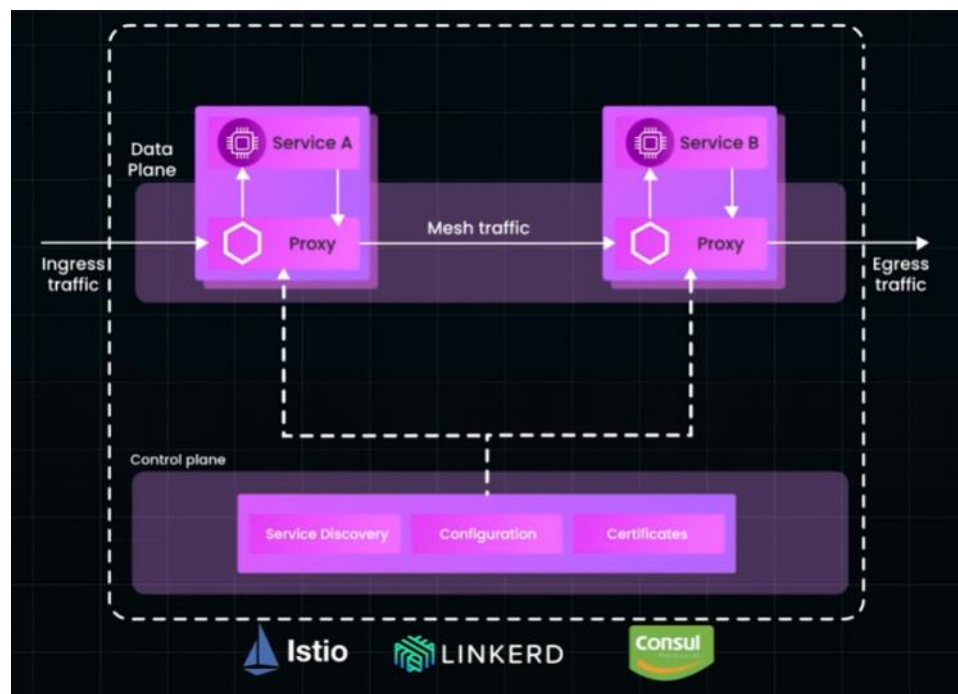
Real-World Use Case: Netflix

- Netflix's architecture includes hundreds of microservices for:
 - Movie and show recommendations
 - User accounts
 - Streaming delivery

- Netflix uses **Istio**, a popular service mesh, to:
- Route traffic based on user location
- Handle failures gracefully ensuring uninterrupted user experience
- Secure inter-service communication with mTLS, safeguarding sensitive user data (viewing history, accounts)
- Deploy new microservice versions without user impact, even during high-traffic events like TV series launches

Popular Service Mesh Tools

Tool Name	Key Features	Use Cases	Integration
Istio	Traffic management, observability, security (mTLS), Canary deployments	Complex microservices, Kubernetes environments	Easily integrates with Kubernetes and existing systems
Linkerd	Lightweight, performance-focused	Simpler microservices architectures	Kubernetes compatible
Consul	Service discovery, health checking, network segmentation, typical service mesh features	Service discovery and health-focused environments	Integrates with multiple platforms



When to Use a Service Mesh

- Recommended when:
- You manage a **large number of microservices** requiring reliable and secure communication
- You need **fine-grained traffic control** (e.g., Canary deployments, A/B testing)
- Security and encryption of internal service traffic is a priority
- You require **deep observability** into service performance and behavior
- Not always necessary for small architectures, as service mesh may introduce **unnecessary complexity** if the microservice count is low.

Conclusion

- For teams building or scaling microservices architectures, a service mesh can be a **critical tool** to:
- Simplify communication management
- Enhance security through automated encryption
- Enable advanced traffic routing and deployment strategies
- Provide comprehensive observability for debugging and performance optimization
- Ultimately, a service mesh helps keep complex microservices ecosystems running **smoothly and securely** as they grow.

SOLID Principles

10 March 2026 18:40

The **SOLID** principles are five foundational guidelines in object-oriented design that make software more maintainable, scalable, and flexible. They were introduced by Robert C. Martin (Uncle Bob) and are widely used in Java, Spring Boot, and modern system design.

Principle	Definition	Example
S – Single Responsibility Principle (SRP)	A class should have only one reason to change (one responsibility).	A <code>ReportPrinter</code> class should only handle printing, not report generation.
O – Open/Closed Principle (OCP)	Classes should be open for extension but closed for modification.	Adding new payment methods via subclasses instead of editing existing code.
L – Liskov Substitution Principle (LSP)	Subtypes must be substitutable for their base types without breaking functionality.	A <code>Square</code> class should behave correctly when used as a <code>Rectangle</code> .
I – Interface Segregation Principle (ISP)	Clients should not be forced to depend on interfaces they don't use.	Instead of one large <code>Vehicle</code> interface, split into <code>Drivable</code> , <code>Flyable</code> , <code>Sailable</code> .
D – Dependency Inversion Principle (DIP)	Depend on abstractions, not concrete implementations.	Use <code>PaymentService</code> interface instead of directly depending on <code>CreditCardPayment</code> .

◆ S — Single Responsibility Principle (SRP)

Definition:

A class should have **only one reason to change**.

✗ Bad

⌞ Java

```
class UserService {  
    void saveUser() { }  
    void sendEmail() { }  
}
```

✓ Good

⌞ Java

```
class UserService {  
    void saveUser() { }  
}  
  
class EmailService {  
    void sendEmail() { }  
}
```

👉 Separate responsibilities → easier to maintain & test

◆ O — Open/Closed Principle (OCP)

Definition:

Software should be open for extension but closed for modification.

✖ Bad

Java

```
class DiscountService {
    double getDiscount(String type) {
        if (type.equals("NEW")) return 10;
        if (type.equals("OLD")) return 5;
        return 0;
    }
}
```

✔ Good

Java

```
interface Discount {
    double apply();
}

class NewCustomerDiscount implements Discount {
    public double apply() { return 10; }
}

class OldCustomerDiscount implements Discount {
    public double apply() { return 5; }
}
```

👉 Add new discount types without changing existing code

◆ L — Liskov Substitution Principle (LSP)

Definition:

Subclasses should be replaceable for their base classes without breaking behavior.

✖ Bad

Java

```
class Bird {  
    void fly() { }  
}  
  
class Ostrich extends Bird {  
    void fly() {  
        throw new UnsupportedOperationException();  
    }  
}
```

✔ Good

Java

```
class Bird { }  
  
class FlyingBird extends Bird {  
    void fly() { }  
}  
  
class Ostrich extends Bird { }
```

👉 Don't force behavior that doesn't apply



◆ I — Interface Segregation Principle (ISP)

Definition:

Clients should not depend on interfaces they don't use.

✖ Bad

```
«» Java
interface Worker {
    void work();
    void eat();
}
```

✔ Good

```
«» Java
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}
```

👉 Smaller, focused interfaces

◆ D — Dependency Inversion Principle (DIP)

Definition:

Depend on **abstractions**, not concrete implementations.

✖ Bad

🔗 Java

```
class OrderService {  
    private MySQLDatabase db = new MySQLDatabase();  
}
```

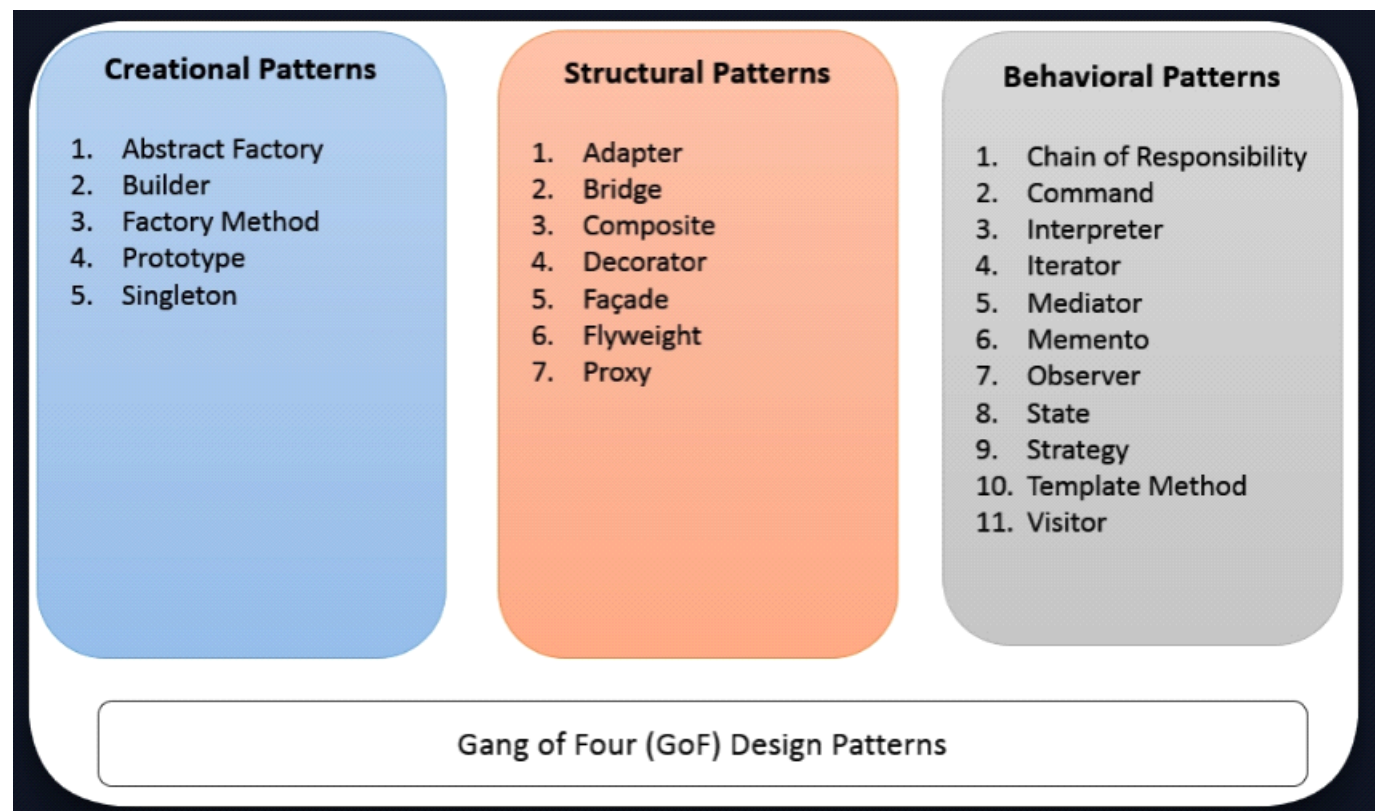
✔ Good

🔗 Java

```
interface Database {  
    void save();  
}  
  
class MySQLDatabase implements Database {  
    public void save() { }  
}  
  
class OrderService {  
    private Database db;  
  
    OrderService(Database db) {  
        this.db = db;  
    }  
}
```

👉 Enables flexibility, testing, and loose coupling





Abstract Factory

18 March 2026 12:21

Builder

18 March 2026

12:21

Factory Method

18 March 2026 12:21

Singleton

18 March 2026 12:22

Prototype

18 March 2026 12:22

Adapter

18 March 2026 12:21

Bridge

18 March 2026

12:22

Composite

18 March 2026 12:22

Decorator

18 March 2026 12:22

Facade

18 March 2026

12:22

Flyyweight

18 March 2026 12:22

Proxy

18 March 2026 12:23

Chain of Responsibility

18 March 2026 12:21

Command

18 March 2026 12:23

Intepreter

18 March 2026 12:23

Iterator

18 March 2026 12:23

Mediator

18 March 2026 12:23

Memento

18 March 2026 12:24

Observer

18 March 2026 12:24

State

18 March 2026 12:24

Strategy

18 March 2026 12:24

Template Method

18 March 2026 12:24

Visitor

18 March 2026

12:24